# LAB 08 — Pretrained Transformers and BERT Fine-Tuning

Bài toán: dùng một Transformer đã pretrain (DistilBERT) cho phân loại sentiment IMDb, so sánh với Transformer train from scratch của LAB 07. Thí nghiệm gồm: frozen so với fine-tuning, lượng training data, max sequence length, và domain shift (IMDb → Yelp / Amazon).

Pipeline: `raw text → WordPiece tokenizer → [CLS] … [SEP] + attention mask → DistilBERT (6 layer, pretrained bằng MLM) → h[CLS] → Dropout → Linear → softmax → positive / negative`.

## Cấu trúc thư mục

| File | Nội dung |
|---|---|
| `report.pdf` | Bản tổng hợp: README, calculations, prediction, 2 notebook, error_analysis, reflection |
| `calculations.md` | Bài 1: MLM loss (−ln 0.70, 0.95, 0.01). Bài 2: tokenize "The cats are playing." (token, ID, special token, attention mask, đếm word / token / subword) |
| `prediction.md` | Prediction 1–3 (và dự đoán phụ cho Exp 4, 5), ghi trước khi chạy |
| `tokenization.ipynb` | `tokens.keys()`, `input_ids` / `attention_mask`, padding trong batch, kiểm tra tác dụng của attention mask, subword (từ hiếm, tên riêng, lỗi chính tả, tiếng Việt, emoji), DistilBERT điền `[MASK]` và kiểm tra Bài 1 |
| `pretrained_classification.py` | Dữ liệu (IMDb chia như LAB 05–07, Yelp / Amazon), `PretrainedClassifier` (encoder → [CLS] → Linear), `ScratchTransformer` (cấu hình LAB 07, cùng tokenizer), `train_model`, frozen mode (`extract_features` + `train_head`), `predict_proba`, `metrics` |
| `fine_tuning.ipynb` | Exp 1–5, application, confidence analysis, error analysis (đã chạy, có output) |
| `results.csv` | Mọi số liệu (dạng dài: experiment, setting, metric, value) |
| `images/data_efficiency.png` | Accuracy / F1 theo 10–100% data: pretrained, frozen ([CLS], mean), scratch |
| `images/confusion_matrix.png` | Confusion matrix của scratch, frozen, fine-tuned |
| `images/confidence_analysis.png` | Reliability diagram (3 model), phân bố confidence của dự đoán đúng / sai |
| `images/learning_curves.png` | Train / validation loss, validation accuracy của scratch và fine-tuned |
| `images/sequence_length.png` | Accuracy, thời gian train, bộ nhớ GPU theo T = 64 / 128 / 256 |
| `error_analysis.md` | Metrics, confusion matrix, câu tự viết, 4 FP + 4 FN + 4 lỗi tự tin nhất, phân loại lỗi, confidence analysis |
| `reflection.md` | Đối chiếu prediction, trả lời câu hỏi từng experiment, Q1–Q6, learning check, reflection |

## Reproducibility

| | |
|---|---|
| Model | `distilbert-base-uncased` (66.4 triệu tham số) + head Linear(768, 2). Baseline: Transformer from scratch (cấu hình LAB 07: d_model 128, 4 head, d_ff 256, 1 layer, mean pooling; 4.0 triệu tham số) |
| Tokenizer | `distilbert-base-uncased` (WordPiece, vocab 30 522, chữ thường), dùng cho cả hai model |
| Dataset | IMDb (`stanfordnlp/imdb`); domain shift: 2 000 mẫu cân bằng nhãn từ test của `fancyzhx/yelp_polarity` và `fancyzhx/amazon_polarity` |
| Train / validation / test split | 19 923 / 4 981 (bỏ review trùng, chia stratified 80/20, seed 42) / 25 000. Giống LAB 05–07 |

| | |
|---|---|
| Max sequence length | 256 (Exp 4: 64 / 128 / 256) |
| Batch size | 16 (pretrained), 64 (scratch) |
| Learning rate | 2e-5 (pretrained), 1e-3 (scratch); warmup 6% rồi giảm tuyến tính; head của frozen: Adam 1e-3 |
| Epochs | 2 (pretrained), 6 (scratch), 30 (head của frozen); chọn epoch có validation loss thấp nhất |
| Optimizer | AdamW, weight decay 0.01, clip gradient 1.0, bf16 autocast |
| Random seed | 42, 43, 44 cho Exp 1; 42 cho các thí nghiệm còn lại; subset 10/25/50% lồng nhau (seed 42) |
| Hardware | NVIDIA RTX 3050 Laptop (4 GB), PyTorch 2.6.0 + CUDA 12.4, transformers 5.17.0 |
| Training time | Fine-tuning 484 s / 2 epoch (T = 256); scratch 33 s / 6 epoch; frozen 290 s (chủ yếu để trích feature). Cả notebook: 114 phút |

## Kết quả chính

| Model (IMDb test) | Params train được | Train time | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|---|---|
| Transformer from scratch (3 seed) | 4 039 554 | 33 s | 0.835 ± 0.004 | 0.835 | 0.836 | 0.835 ± 0.003 |
| Frozen DistilBERT, [CLS] | 1 538 | 290 s | 0.853 | 0.867 | 0.833 | 0.850 |
| Frozen DistilBERT, mean | 1 538 | 289 s | 0.859 | 0.873 | 0.839 | 0.856 |
| **DistilBERT fine-tuned (3 seed)** | 66 364 418 | 484 s | **0.911 ± 0.002** | 0.909 | 0.915 | **0.911 ± 0.002** |

| Training data | 10% | 25% | 50% | 100% |
|---|---|---|---|---|
| Fine-tuned | 0.876 | 0.894 | 0.897 | 0.911 |
| Frozen (mean) | 0.837 | 0.849 | 0.854 | 0.859 |
| Scratch | 0.712 | 0.778 | 0.813 | 0.835 |

| Max length | 64 | 128 | 256 |
|---|---|---|---|
| Accuracy | 0.822 | 0.875 | 0.910 |
| Train time / peak GPU memory | 196 s / 1 345 MB | 282 s / 1 548 MB | 484 s / 1 951 MB |

| Accuracy | IMDb | Amazon | Yelp |
|---|---|---|---|
| Scratch | 0.832 | 0.785 | 0.796 |
| Fine-tuned | 0.910 | 0.927 | 0.924 |

Các phát hiện chính:

1. **Pretrained + fine-tuning hơn from scratch 7.6 điểm** (0.911 so với 0.835, ổn định qua 3 seed). Đổi lại, model lớn hơn 16 lần và train lâu hơn khoảng 15 lần.
2. **Frozen encoder đã vượt from scratch:** chỉ train 1 538 tham số mà đạt 0.853–0.859. Fine-tuning vẫn hơn frozen 5–6 điểm.
3. **Data efficiency:** - Chỉ với 10% data, pretrained (0.876) đã hơn from scratch dùng 100% data (0.835). - Đi từ 100% xuống 10%, pretrained giảm 3.5 điểm, from scratch giảm 12.4 điểm.
4. **Max length:** T = 64 → 256 tăng 8.8 điểm accuracy vì review IMDb dài (median 221 token). Chi phí tăng chậm hơn $T^2$ ở mức T này (thời gian × 2.5, bộ nhớ × 1.45).
5. **Domain shift:** - Model fine-tuned không giảm trên Yelp và Amazon (review ngắn, cảm xúc nói thẳng). From scratch giảm 3.6–4.7 điểm. - Đây là domain gần; kết quả không chứng minh model "hiểu mọi domain".

6. **Lỗi:** - Review bị cắt ở 256 token có tỉ lệ lỗi 12.4%, gấp đôi review không bị cắt (6.7%). - 3 trên 4 lỗi tự tin nhất là review khen rõ ràng nhưng mang nhãn negative (label noise). - Nhóm confidence ≥ 0.95 đúng 98.1% nhưng vẫn chiếm 14% tổng số lỗi.

## Lưu ý

- **Best epoch:** model fine-tuned đạt validation loss thấp nhất ở epoch 1 trong 7 trên 8 lần chạy. Epoch 2 làm val loss tăng (0.238 → 0.279) dù val accuracy vẫn tăng nhẹ.
- **Bước bị bỏ qua (NaN):**
- Như LAB 07, GPU này thỉnh thoảng cho ra NaN; bước có gradient không hữu hạn bị bỏ qua.
- Tổng cộng 85 bước: 63 / 2 490 bước ở T = 128 (2.5%), 21 ở T = 64, 1 ở seed 43. Các lần chạy T = 256 còn lại không có.
- Kết quả T = 64 / 128 vì vậy có thể thấp hơn một chút so với khi train đủ bước.
- **Thời gian:** wall-clock của seed 43 epoch 1 (1 678 s) và thời gian dự đoán test ở T = 256 (232 s) bị kéo dài do máy chững lại khi đang chạy. Cột train time chỉ tính thời gian các bước train.
- **Số seed:** các điểm 10–50% data, T = 64 / 128 và domain shift chỉ chạy 1 seed (42).

## Cách chạy lại

```
pip install torch transformers datasets scikit-learn pandas matplotlib nbconvert ipykernel
cd lab8
python pretrained_classification.py      # kiểm tra nhanh: tokenizer, số tham số, forward
python -m jupyter nbconvert --to notebook --execute --inplace tokenization.ipynb
python -m jupyter nbconvert --to notebook --execute --inplace --ExecutePreprocessor.timeout=-1
fine_tuning.ipynb   # khoảng 2 giờ trên RTX 3050 Laptop
```

Đặt biến môi trường `LAB8_QUICK=1` để chạy thử notebook trên dữ liệu nhỏ (khoảng 6 phút). Đặt `LAB8_LOG=<file>` để ghi tiến độ ra file.

## AI assistance statement

- **AI-assisted code:** Claude Code.
- **What was generated:**
- `pretrained_classification.py` (data loader, model, training loop, frozen mode);
- code của 2 notebook (thí nghiệm, bảng, biểu đồ);
- bản nháp các file markdown;
- ghép `report.pdf`.
- **What was modified:**
- Baseline from scratch ban đầu dùng pooling [CLS] và nhân embedding với √d, chỉ đạt khoảng 0.68. Khi kiểm tra thấy model bị underfit; chuyển sang mean pooling, bỏ hệ số √d như LAB 07 thì đạt 0.835, ngang LAB 07.
- Đo bộ nhớ GPU ban đầu tính cả model khác đang nằm trên GPU; sửa lại để chỉ đo phần của lần train đang chạy.
- Thêm frozen mean pooling và thêm from scratch vào Exp 3 và Exp 5 để so sánh.
- Domain shift chỉ tải file test của Yelp và Amazon thay vì cả dataset.
- **What was verified:**
- Bài tính tay MLM khớp `nll_loss` của PyTorch;
- tokenization "The cats are playing." khớp output tokenizer;
- attention mask đúng thì padding không làm đổi vector [CLS] (sai số 7e-6);
- số liệu trong các file `.md` lấy từ output notebook.

- **Student:** MLM calculation, prediction (ghi trước khi chạy), giải thích pretraining / fine-tuning, phân tích kết quả, error analysis, reflection, learning check.

Thứ ngày . .

Bài 1:

Câu The cat [Mask] fish., token đúng là eats.
Model dự đoán cat 0,1, dog 0,05, eats 0.7, fish 0.15.
L = - ln P(eats) = - ln 0.7 = 0,3567

| P(eats) | 0.95 | 0.7 | 0.01 |
|---|---|---|---|
| L | 0,0513 | 0.3567 | 4,6052 |
| so với P=0.7 | giảm 7 lần | | tăng 13 lần |

- Vì sao cross entropy phù hợp vs MLM: MLM là phân loại trên toàn vocabulary, mỗi vị trí bị mask có đúng 1 token thật nên Log P chính là loss đúng nghĩa. Nó phạt nặng khi model tự tin mà sai (0,01 -> 4.6) và gần như ko phạt khi đúng (0.95 -> 0.05) gradient đơn giản $\partial L / \partial z_k = p_k - 1[k = y]$

Bài 2.

Tokenizer, distilbert-base-uncased (WordPiece, chữ thường). Token sequence, token ID, special token và attention mask.

| Vị trí | 0 | 1 | 2 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| token | [CLS] | the | cats | playing | - | are | [SEP] |
| token ID | 101 | 1996 | 8870 | 2652 | 1011 | 2024 | 102 |
| attention mask | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

- special token [CLS] (101) ở đầu - vector của nó dùng làm representation của cả câu khi classification, [SEP] (102) ở cuối đánh dấu kết câu
- attention mask: toàn 1 vì câu đứng 1 mình. Nếu cùng batch với câu dài 9 token câu này phải thêm 2 [pad] (ID 0) và mask thành [1, 1, 1, 1, 1, 1, 1, 0, 0]


HONG HA
LAB 08 — Pretrained Transformers and BERT Fine-Tuning — Trang 5/34
Scanned with CamScanner

Thứ ngày . .

| | số lượng | ghi chú |
|---|---|---|
| Words | 4 | The/cat/are/playing |
| Token | 7 | thêm [CLS], [SEP] |
| Token (bỏ special) | 5 | dấu "." tách khỏi playing |
| Subword | 0 | cả 4 từ đều có sẵn trong vocab |

Nếu nhiều token hơn word là do special token và dấu câu tách riêng, không phải do subword. Ví dụ: "cats" và "playing" là token nguyên. Subword chỉ xuất hiện với từ ít gặp, ví dụ: unwatchable → un ## watch ## able.



HONG HA



Scanned with CamScanner

# Prediction — LAB 08

Ghi trước khi chạy bất kỳ thí nghiệm nào.

## Prediction 1 — Pretrained + fine-tuning so với Transformer from scratch

- **Prediction:** pretrained (DistilBERT) fine-tuned tốt hơn rõ rệt: accuracy khoảng 0.90–0.92, so với khoảng 0.83–0.85 của Transformer from scratch (LAB 07 đạt 0.835).
- **Reason:** model from scratch phải học cả nghĩa từ lẫn task chỉ từ khoảng 20 000 review. Model pretrained đã có contextual representation học từ corpus rất lớn, nên phần cần học thêm chỉ là ánh xạ sang sentiment.
- **Expected evidence:**
- accuracy và F1 trên test chênh ít nhất 5 điểm;
- validation loss của model pretrained thấp ngay từ epoch 1;
- đổi lại, train mỗi epoch chậm hơn nhiều vì model có khoảng 66 triệu tham số, so với khoảng 4 triệu.

## Prediction 2 — Freeze encoder, chỉ train classification head

- **Chọn: C. kém hơn fine-tuning toàn bộ.**
- **Giải thích:**
- Frozen vẫn phân loại được, vì vector [CLS] / mean của encoder đã chứa thông tin về nghĩa của câu, mà sentiment là một phần thông tin đó. Dự đoán accuracy khoảng 0.80–0.85.
- Tuy nhiên encoder được pretrain cho MLM, không phải cho sentiment, nên các đặc trưng cần cho sentiment không nằm sẵn theo một hướng tuyến tính rõ ràng. Fine-tuning cho phép điều chỉnh lại representation, nên dự đoán hơn khoảng 5–8 điểm.
- Ngược lại, frozen train rất nhanh, vì chỉ cần chạy encoder một lần để lấy feature.

## Prediction 3 — Giảm training data xuống 10%

- **Prediction:** model from scratch giảm mạnh hơn. Ở 10% (khoảng 2 000 review), from scratch có thể xuống khoảng 0.70, còn pretrained vẫn khoảng 0.86–0.88.
- **Cơ sở:**
- Pretraining đã cung cấp sẵn kiến thức về từ, cú pháp và ngữ cảnh. Với dữ liệu downstream ít, model pretrained chỉ cần học "hướng" sentiment trong không gian representation có sẵn.
- Model from scratch phải học embedding của hơn 30 000 token từ đầu. Với 2 000 review, phần lớn token chỉ xuất hiện vài lần, nên embedding gần như ngẫu nhiên và model dễ overfit.
- Do đó đường data-efficiency của pretrained sẽ phẳng hơn: khoảng cách giữa 10% và 100% nhỏ hơn.

## Dự đoán phụ (Exp 4, Exp 5)

- **Max length 64 / 128 / 256:** accuracy tăng theo T, vì review IMDb dài (trung vị khoảng 230 từ), nhưng mức tăng nhỏ dần. Thời gian và bộ nhớ tăng nhanh hơn tuyến tính vì attention có chi phí $O(T^2)$.
- **Domain shift (train IMDb → test Yelp / Amazon):** accuracy giảm nhưng không sụp đổ, khoảng 0.85–0.88, vì từ chỉ cảm xúc ("great", "terrible") là chung giữa các domain. From scratch sẽ giảm mạnh hơn, vì vocabulary học từ phim không phủ được từ về đồ ăn và sản phẩm.

# LAB 08 — Tokenization, special tokens, attention mask và MLM

Tokenizer: `distilbert-base-uncased` (WordPiece, vocab 30 522, chữ thường), cùng tokenizer dùng trong `fine_tuning.ipynb`.

1. Calculation Exercise 2: "The cats are playing." → token, ID, special token, attention mask, đếm word / token / subword;
2. `tokens.keys()`, padding trong batch và attention mask;
3. subword tokenization với từ dài, từ hiếm, lỗi chính tả, tiếng Việt, emoji;
4. Masked Language Modeling: DistilBERT dự đoán "The cat [MASK] fish." và kiểm tra lại Calculation Exercise 1.

```
import os, warnings
os.environ["HF_HUB_DISABLE_PROGRESS_BARS"] = "1"; os.environ["TRANSFORMERS_VERBOSITY"] = "error"
warnings.filterwarnings("ignore")
import math
import pandas as pd, torch
from transformers import AutoTokenizer, AutoModelForMaskedLM
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
print(type(tokenizer).__name__, "| vocab size:", tokenizer.vocab_size, "| model_max_length:",
tokenizer.model_max_length)
print("special tokens:", {t: tokenizer.convert_tokens_to_ids(t) for t in tokenizer.all_special_tokens})
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits
and faster downloads.
BertTokenizer | vocab size: 30522 | model_max_length: 512
special tokens: {'[UNK]': 100, '[SEP]': 102, '[PAD]': 0, '[CLS]': 101, '[MASK]': 103}
```

# 1. Calculation Exercise 2 — "The cats are playing."

```
text = "The cats are playing."
tokens = tokenizer(text, padding=True, truncation=True, return_tensors="pt")
print("keys:", list(tokens.keys()))
ids = tokens["input_ids"][0].tolist()
toks = tokenizer.convert_ids_to_tokens(ids)
special = set(tokenizer.all_special_ids)
pd.DataFrame({"position": range(len(ids)), "token": toks, "token ID": ids,
              "special?": [i in special for i in ids],
              "attention_mask": tokens["attention_mask"][0].tolist()}).set_index("position")
```

```
keys: ['input_ids', 'token_type_ids', 'attention_mask']
```

| position | token | token ID | special? | attention_mask |
|---|---|---|---|---|
| **0** | [CLS] | 101 | True | 1 |
| **1** | the | 1996 | False | 1 |
| **2** | cats | 8870 | False | 1 |
| **3** | are | 2024 | False | 1 |
| **4** | playing | 2652 | False | 1 |
| **5** | . | 1012 | False | 1 |
| **6** | [SEP] | 102 | True | 1 |

```
words = text.split()
content = [t for t, i in zip(toks, ids) if i not in special]
subword = [t for t in content if t.startswith("##")]
print("words (tách theo khoảng trắng):", len(words), words)
print("tokens (cả special):          ", len(toks))
print("tokens (bỏ special):          ", len(content), content)
print("subword tokens (bắt đầu ##):  ", len(subword), subword)
print("decode lại:", tokenizer.decode(ids, skip_special_tokens=True))
```

```
words (tách theo khoảng trắng): 4 ['The', 'cats', 'are', 'playing.']
tokens (cả special):           7
tokens (bỏ special):           5 ['the', 'cats', 'are', 'playing', '.']
subword tokens (bắt đầu ##):   0 []
decode lại: the cats are playing.
```

Mỗi từ ở đây đều có sẵn trong vocabulary nên **không bị tách thành subword**: "cats" và "playing" là token nguyên, không thành `cat + ##s` hay `play + ##ing`. Dấu "." bị tách khỏi "playing" thành token riêng, nên 4 word thành 5 token nội dung, cộng `[CLS]` và `[SEP]` là 7 token.

## 2. Batch: padding và attention mask

Khi các câu trong batch dài khác nhau, câu ngắn được thêm `[PAD]` (ID 0) cho bằng câu dài nhất. Attention mask = 0 ở các vị trí đó.

```
batch = ["The cat eats fish.", "The cats are playing.", "This movie is great"]
enc = tokenizer(batch, padding=True, truncation=True, return_tensors="pt")
for k, v in enc.items():
    print(k, tuple(v.shape))
for i in range(len(batch)):
    print(f"{batch[i]!r:26}", tokenizer.convert_ids_to_tokens(enc["input_ids"][i]), enc["attention_mask"]
[i].tolist())
```

```
input_ids (3, 7)
token_type_ids (3, 7)
attention_mask (3, 7)
'The cat eats fish.'       ['[CLS]', 'the', 'cat', 'eats', 'fish', '.', '[SEP]'] [1, 1, 1, 1, 1, 1, 1]
'The cats are playing.'    ['[CLS]', 'the', 'cats', 'are', 'playing', '.', '[SEP]'] [1, 1, 1, 1, 1, 1, 1]
'This movie is great'      ['[CLS]', 'this', 'movie', 'is', 'great', '[SEP]', '[PAD]'] [1, 1, 1, 1, 1, 1, 0]
```

`token_type_ids` (segment ID) dùng cho cặp câu `[CLS] A [SEP] B [SEP]`. DistilBERT không dùng nó, nên model trong `pretrained_classification.py` chỉ nhận `input_ids` và `attention_mask`.

```
pair = tokenizer("The cat eats fish.", "It is hungry.")
print(tokenizer.convert_ids_to_tokens(pair["input_ids"]))
print("token_type_ids:", pair["token_type_ids"])
```

```
['[CLS]', 'the', 'cat', 'eats', 'fish', '.', '[SEP]', 'it', 'is', 'hungry', '.', '[SEP]']
token_type_ids: [0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
```

**Attention mask có tác dụng thật không?** Cùng một câu, tính representation [CLS] khi đứng một mình, khi bị pad thêm 10 token mà có mask, và khi pad mà mask toàn 1.

```
from transformers import AutoModel
enc_model = AutoModel.from_pretrained("distilbert-base-uncased").eval()
one = tokenizer("The cats are playing.", return_tensors="pt")
pad_ids = torch.cat([one["input_ids"], torch.zeros(1, 10, dtype=torch.long)], 1)
good_mask = torch.cat([one["attention_mask"], torch.zeros(1, 10, dtype=torch.long)], 1)
with torch.no_grad():
    h0 = enc_model(**one).last_hidden_state[0, 0]
    h_mask = enc_model(input_ids=pad_ids, attention_mask=good_mask).last_hidden_state[0, 0]
    h_nomask = enc_model(input_ids=pad_ids, attention_mask=torch.ones_like(pad_ids)).last_hidden_state[0, 0]
print(f"‖h[CLS] (pad + mask đúng) − h[CLS] (không pad)‖   = {(h_mask - h0).norm():.2e}")
print(f"‖h[CLS] (pad, mask toàn 1) − h[CLS] (không pad)‖  = {(h_nomask - h0).norm():.2f}   (‖h‖ =
{h0.norm():.2f})")
```

```
‖h[CLS] (pad + mask đúng) − h[CLS] (không pad)‖   = 6.96e-06
‖h[CLS] (pad, mask toàn 1) − h[CLS] (không pad)‖  = 6.68   (‖h‖ = 12.33)
```

Có mask đúng thì padding không làm đổi representation (sai số làm tròn, khoảng 1e-6). Bỏ mask thì `[PAD]` được attend như nội dung thật và vector [CLS] đổi hẳn.

# 3. Subword tokenization

```
examples = ["unbelievable", "tokenization", "playing", "unwatchable", "overacting", "cinematography",
            "Tarantino", "Schwarzenegger", "soooo baaaad", "gr8 movie", "COVID-19",
            "phim rất hay", "Phở", "😍", "naïve café"]
rows = []
for w in examples:
    t = tokenizer.tokenize(w)
    rows.append({"input": w, "tokens": " ".join(t), "#words": len(w.split()), "#tokens": len(t),
                 "#subword (##)": sum(x.startswith("##") for x in t), "[UNK]?": "[UNK]" in t})
pd.DataFrame(rows).set_index("input")
```

| input | tokens | #words | #tokens | #subword (##) | [UNK]? |
|---|---|---|---|---|---|
| **unbelievable** | unbelievable | 1 | 1 | 0 | False |
| **tokenization** | token ##ization | 1 | 2 | 1 | False |
| **playing** | playing | 1 | 1 | 0 | False |
| **unwatchable** | un ##watch ##able | 1 | 3 | 2 | False |
| **overacting** | over ##act ##ing | 1 | 3 | 2 | False |
| **cinematography** | cinematography | 1 | 1 | 0 | False |
| **Tarantino** | tara ##ntino | 1 | 2 | 1 | False |
| **Schwarzenegger** | schwarz ##ene ##gger | 1 | 3 | 2 | False |
| **soooo baaaad** | soo ##oo ba ##aa ##ad | 2 | 5 | 3 | False |
| **gr8 movie** | gr ##8 movie | 2 | 3 | 1 | False |
| **COVID-19** | co ##vid - 19 | 1 | 4 | 1 | False |
| **phim rất hay** | phi ##m rat hay | 3 | 4 | 1 | False |
| **Phở** | ph ##o | 1 | 2 | 1 | False |
| 😍 | [UNK] | 1 | 1 | 0 | True |
| **naïve café** | naive cafe | 2 | 2 | 0 | False |

Nhận xét:

- **Từ phổ biến** ("playing", "unbelievable", "cinematography") là một token nguyên. Ví dụ `un + believe + able` trong đề chỉ là minh họa; WordPiece của DistilBERT không tách từ này.
- **Từ ít gặp** được tách theo mảnh có nghĩa: "unwatchable" thành `un ##watch ##able`, "overacting" thành `over ##act ##ing`. Model vẫn nhận ra gốc "watch" / "act" dù chưa thấy nguyên từ.
- **Tên riêng** bị tách thành mảnh vô nghĩa ("tara ##ntino", "schwarz ##ene ##gger"). Dù vậy không có từ nào là out-of-vocabulary như LAB 05–07 (ở đó từ ngoài top 25 000 thành `<unk>`).
- **Kéo dài chữ hoặc viết tắt** ("soooo baaaad", "gr8") bị cắt vụn. Nghĩa phải ghép lại từ nhiều mảnh không mang nghĩa.
- **Tiếng Việt:** tokenizer `uncased` bỏ dấu ("rất" thành "rat", "phở" thành `ph ##o`), nên mất thông tin thanh điệu. Emoji 😍 không có trong vocab nên thành `[UNK]`, tức mất hẳn một tín hiệu sentiment.

Đây là *tokenizer limitation*: tokenizer cố định từ lúc pretrain, không thích nghi với domain hay ngôn ngữ mới.

# 4. Masked Language Modeling

```
mlm = AutoModelForMaskedLM.from_pretrained("distilbert-base-uncased").eval()
def fill_mask(sentence, candidates=(), k=8):
    enc = tokenizer(sentence, return_tensors="pt")
    pos = (enc["input_ids"][0] == tokenizer.mask_token_id).nonzero()[0, 0]
    with torch.no_grad():
        prob = torch.softmax(mlm(**enc).logits[0, pos], -1)
    top = torch.topk(prob, k)
    print(sentence)
    print("  top-k:", ", ".join(f"{tokenizer.convert_ids_to_tokens(int(i))} {p:.3f}" for p, i in zip(top.values,
top.indices)))
    for c in candidates:
        p = prob[tokenizer.convert_tokens_to_ids(c)].item()
        print(f"  P({c}) = {p:.4f} → loss −log P = {-math.log(p):.3f}")
    return prob

_ = fill_mask("The cat [MASK] fish.", ["eats", "dog", "cat", "fish"])
_ = fill_mask("This movie is [MASK]. I loved every minute of it.", ["great", "terrible"])
_ = fill_mask("This movie is [MASK]. I walked out after twenty minutes.", ["great", "terrible"])
```

```
The cat [MASK] fish.
  top-k: eats 0.351, ##tail 0.073, attracts 0.065, ate 0.045, hunts 0.031, loves 0.028, eat 0.015, hates 0.014
  P(eats) = 0.3505 → loss −log P = 1.048
  P(dog) = 0.0002 → loss −log P = 8.594
  P(cat) = 0.0000 → loss −log P = 11.711
  P(fish) = 0.0001 → loss −log P = 9.820
This movie is [MASK]. I loved every minute of it.
  top-k: amazing 0.133, incredible 0.090, epic 0.053, awesome 0.047, fantastic 0.042, unbelievable 0.041, perfect
0.036, hilarious 0.028
  P(great) = 0.0178 → loss −log P = 4.031
  P(terrible) = 0.0075 → loss −log P = 4.895
This movie is [MASK]. I walked out after twenty minutes.
  top-k: boring 0.061, awesome 0.030, bad 0.022, awful 0.021, scary 0.019, amazing 0.019, terrible 0.018, hilarious
0.017
  P(great) = 0.0094 → loss −log P = 4.665
  P(terrible) = 0.0184 → loss −log P = 3.997
```

Model chưa từng được train cho sentiment, nhưng chỉ nhờ MLM đã:

- đoán đúng "eats" (0.35) và các động từ hợp ngữ cảnh ("hunts", "loves"), còn "dog" / "cat" / "fish" gần 0;
- đặt top-k toàn từ tích cực ("amazing", "incredible") khi phần sau là "I loved every minute", và chuyển sang "boring", "bad", "awful" khi phần sau là "I walked out". P(great) / P(terrible) đảo chiều giữa hai câu.

Đây là bằng chứng representation pretrained đã chứa thông tin sentiment. Đó là lý do frozen encoder vẫn phân loại được (Exp 2).

## Kiểm tra Calculation Exercise 1

L = −log P(y), log tự nhiên.

```
for p in [0.70, 0.95, 0.01]:
    print(f"P(eats) = {p:.2f} → L = -ln({p}) = {-math.log(p):.4f}")
# phân phối của đề: cross-entropy với one-hot "eats" chỉ lấy đúng xác suất của token đúng
probs = torch.tensor([0.10, 0.05, 0.70, 0.15])
print("F.nll_loss(log p, eats) =", torch.nn.functional.nll_loss(probs.log().unsqueeze(0),
torch.tensor([2])).item())
```

```
P(eats) = 0.70 → L = −ln(0.7) = 0.3567
P(eats) = 0.95 → L = −ln(0.95) = 0.0513
P(eats) = 0.01 → L = −ln(0.01) = 4.6052
F.nll_loss(log p, eats) = 0.3566749691963196
```

# LAB 08 — Fine-tuning pretrained Transformer (DistilBERT) trên IMDb

Code lõi nằm ở `pretrained_classification.py`. Notebook này chạy:

1. dữ liệu IMDb (chia giống LAB 05–07), độ dài review tính bằng token;
2. **Exp 1** — Transformer from scratch so với DistilBERT fine-tuning (3 seed mỗi model);
3. **Exp 2** — frozen encoder (chỉ train head) so với fine-tuning;
4. **Exp 3** — 10 / 25 / 50 / 100% training data (pretrained, frozen, from scratch);
5. **Exp 4** — max length 64 / 128 / 256;
6. **Exp 5** — domain shift: train IMDb, test Yelp và Amazon;
7. application (metrics + confusion matrix), confidence analysis, error analysis.

```
import os, warnings
os.environ["HF_HUB_DISABLE_PROGRESS_BARS"] = "1"; os.environ["TRANSFORMERS_VERBOSITY"] = "error"
os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
warnings.filterwarnings("ignore")
import time, gc
from matplotlib.ticker import NullLocator
import numpy as np, pandas as pd, matplotlib.pyplot as plt, torch
from sklearn.metrics import confusion_matrix
import transformers; transformers.logging.set_verbosity_error()
import pretrained_classification as pc
os.makedirs("images", exist_ok=True)
QUICK = os.environ.get("LAB8_QUICK") == "1"          # chạy thử nhanh trên dữ liệu nhỏ
if os.environ.get("LAB8_LOG"):                       # ghi thêm mọi dòng print ra file để theo dõi tiến độ
    import builtins
    _print = builtins.print
    def _tee(*a, **k):
        _print(*a, **k)
        with open(os.environ["LAB8_LOG"], "a", encoding="utf-8") as f:
            f.write(time.strftime("%H:%M:%S ") + " ".join(map(str, a)) + "\n")
    builtins.print = _tee
SEEDS = [42, 43, 44]
print("torch", torch.__version__, "| transformers", transformers.__version__, "|", pc.DEVICE,
      torch.cuda.get_device_name() if torch.cuda.is_available() else "")

RESULTS = []
def log(exp, setting, metric, value):
    RESULTS.append({"experiment": exp, "setting": setting, "metric": metric, "value": value})
```

```
torch 2.6.0+cu124 | transformers 5.17.0 | cuda NVIDIA GeForce RTX 3050 Laptop GPU
```

## 1. Dữ liệu

Train 19 923 / validation 4 981 (bỏ review trùng, chia stratified, seed 42) / test 25 000, giống LAB 05–07. Văn bản chỉ bỏ thẻ `<br />`; còn lại do tokenizer của DistilBERT xử lý (WordPiece, chữ thường).

```
data = pc.load_imdb()
if QUICK:
    data = {k: d.sample(n, random_state=0).reset_index(drop=True) for (k, d), n in zip(data.items(), [2000, 500,
1000])}
tokenizer = pc.get_tokenizer()
t0 = time.time()
ids = {k: pc.encode(tokenizer, d.text) for k, d in data.items()}
y = {k: d.label.values for k, d in data.items()}
print({k: len(v) for k, v in ids.items()}, f"| tokenize {time.time() - t0:.0f}s")
print("ti lệ positive:", {k: round(v.mean(), 3) for k, v in y.items()})
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
{'train': 19923, 'val': 4981, 'test': 25000} | tokenize 8s
tỉ lệ positive: {'train': np.float64(0.501), 'val': np.float64(0.501), 'test': np.float64(0.5)}
```

```
n_tok = np.array([len(x) for x in ids["train"]])       # đã cắt ở 512
n_word = data["train"].text.str.split().map(len).values
print(f"word / review: median {np.median(n_word):.0f}, mean {n_word.mean():.0f}")
print(f"token / review (có [CLS], [SEP], cắt ở 512): median {np.median(n_tok):.0f}, mean {n_tok.mean():.0f}")
print(f"token / word ≈ {np.median(n_tok / n_word):.2f}")
for T in [64, 128, 256, 512]:
    print(f"T = {T:3}: {np.mean(n_tok > T):.1%} review bị cắt")
    log("data", f"T={T}", "fraction truncated", float(np.mean(n_tok > T)))
```

```
word / review: median 172, mean 231
token / review (có [CLS], [SEP], cắt ở 512): median 221, mean 265
token / word ≈ 1.26
T =  64: 97.9% review bị cắt
T = 128: 87.8% review bị cắt
T = 256: 41.2% review bị cắt
T = 512: 0.0% review bị cắt
```

# 2. Hàm chạy thí nghiệm

- **Pretrained (fine-tuning):** DistilBERT + head [CLS] → Linear. AdamW lr 2e-5, weight decay 0.01, warmup 6% rồi giảm tuyến tính, batch 16, 2 epoch, bf16 autocast.
- **From scratch:** cấu hình LAB 07 (d_model 128, 4 head, d_ff 256, 1 layer, mean pooling trên token thật) nhưng dùng **cùng tokenizer và cùng max length** với model pretrained. AdamW lr 1e-3, batch 64, 6 epoch.
- Cả hai: clip gradient 1.0, chọn epoch có validation loss thấp nhất, đánh giá trên toàn bộ 25 000 review test.
- Như LAB 07, lỗi CUDA chập chờn của GPU này → train lại từ đầu với cùng seed; bước có gradient NaN → bỏ qua (đếm trong `skipped_steps`).

```
FT = dict(epochs=2, bs=16, lr=2e-5)
SCRATCH = dict(epochs=6, bs=64, lr=1e-3)
N_TRAIN = len(ids["train"])
perm = np.random.default_rng(pc.SEED).permutation(N_TRAIN)      # subset lồng nhau: 10% ⊂ 25% ⊂ 50% ⊂ 100%
def subset(frac):
    return np.sort(perm[: int(round(frac * N_TRAIN))])

def run(kind, seed=42, frac=1.0, T=256, keep=False):
    for attempt in range(3):
        try:
            return _run(kind, seed, frac, T, keep)
        except RuntimeError as e:
            if not any(s in str(e) for s in ("CUDA", "cuDNN", "CUBLAS", "cublas")):
                raise
            print(f"  [lỗi GPU: {str(e)[:60]}… → train lại]")
            log("GPU retries", f"{kind} seed={seed} frac={frac} T={T}", "retry", attempt + 1)
            gc.collect(); torch.cuda.empty_cache()
    raise RuntimeError("GPU lỗi 3 lần liên tiếp")

def _run(kind, seed, frac, T, keep):
    pc.set_seed(seed)
    model = pc.PretrainedClassifier() if kind == "pretrained" else pc.ScratchTransformer(tokenizer.vocab_size,
max_len=512)
    cfg = FT if kind == "pretrained" else SCRATCH
    idx = subset(frac)
    tr_ids, tr_y = [ids["train"][i] for i in idx], y["train"][idx]
    print(f"{kind} | seed {seed} | {frac:.0%} train ({len(idx)}) | T = {T}")
    hist, info = pc.train_model(model, tr_ids, tr_y, ids["val"], y["val"], max_len=T, seed=seed, **cfg)
    t0 = time.time()
    p = pc.predict_proba(model, ids["test"], T)
    row = {"model": kind, "seed": seed, "frac": frac, "T": T, "n_train": len(idx), "params":
pc.count_params(model),
```

```
            **info, "test_time": time.time() - t0, **pc.metrics(y["test"], p), "hist": hist}
    print(f"  → test acc {row['accuracy']:.4f} | F1 {row['f1']:.4f} | train {info['train_time']:.0f}s | "
          f"peak {info['peak_mem_mb']:.0f} MB | skipped {info['skipped_steps']}")
    setting = f"{kind} seed={seed} frac={frac} T={T}"
    for k in ["accuracy", "precision", "recall", "f1", "train_time", "peak_mem_mb", "params", "best_epoch",
"skipped_steps"]:
        log("runs", setting, k, float(row[k]))
    if keep:
        row["model_obj"], row["proba"] = model, p
    else:
        del model; gc.collect(); torch.cuda.empty_cache()
    return row

def summary(rows, by, cols=("accuracy", "precision", "recall", "f1", "train_time")):
    df = pd.DataFrame([{k: v for k, v in r.items() if k not in ("hist", "model_obj", "proba", "head")} for r in
rows])
    g = df.groupby(by, sort=False)
    out = g[list(cols)].mean().round(4)
    if g.size().max() > 1:
        for c in ("accuracy", "f1"):
            out[c] = [f"{m:.4f} ± {s:.4f}" for m, s in zip(g[c].mean(), g[c].std())]
    out["params"] = g.params.first()
    out["runs"] = g.size()
    return out
```

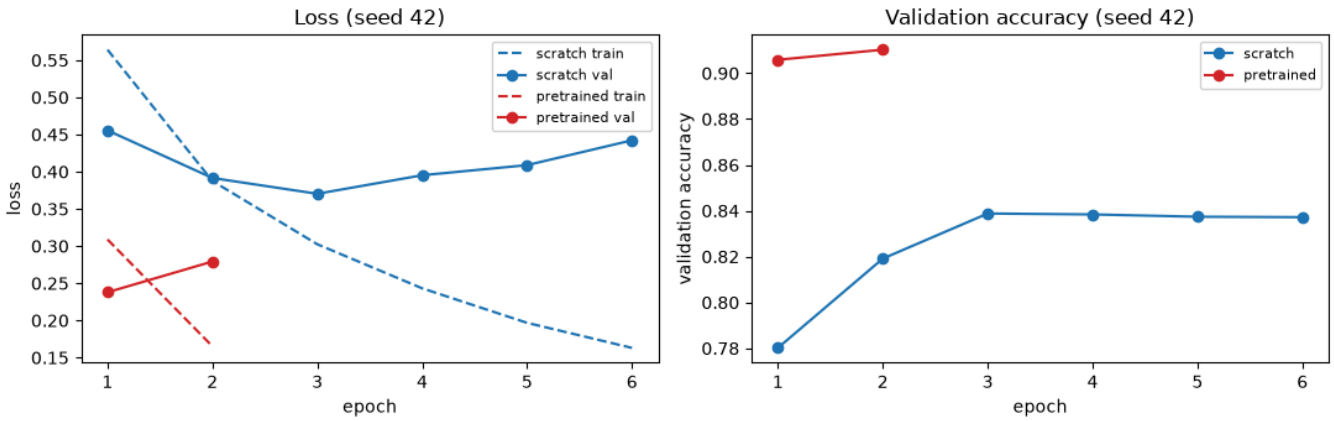
# 3. Exp 1 — Transformer from scratch so với pretrained + fine-tuning (3 seed)

```
t_all = time.time()
exp1 = []
for s in SEEDS:
    exp1.append(run("scratch", seed=s, keep=(s == 42)))
for s in SEEDS:
    exp1.append(run("pretrained", seed=s, keep=(s == 42)))
scratch42 = next(r for r in exp1 if r["model"] == "scratch" and r["seed"] == 42)
ft42 = next(r for r in exp1 if r["model"] == "pretrained" and r["seed"] == 42)
tab1 = summary(exp1, "model")
for m, r in tab1.iterrows():
    for c in ("accuracy", "f1"):
        v = pd.DataFrame([x for x in exp1 if x["model"] == m])[c]
        log("Exp1", m, c + " mean", v.mean()); log("Exp1", m, c + " std", v.std())
    log("Exp1", m, "train_time mean", r.train_time)
tab1
```

```
scratch | seed 42 | 100% train (19923) | T = 256
  epoch 1: train loss 0.5641 | val loss 0.4556 | val acc 0.7802 | 7s
  epoch 2: train loss 0.3876 | val loss 0.3916 | val acc 0.8191 | 6s
  epoch 3: train loss 0.3024 | val loss 0.3703 | val acc 0.8388 | 6s
  epoch 4: train loss 0.2432 | val loss 0.3953 | val acc 0.8384 | 6s
  epoch 5: train loss 0.1968 | val loss 0.4089 | val acc 0.8374 | 6s
  epoch 6: train loss 0.1631 | val loss 0.4422 | val acc 0.8372 | 6s
  → test acc 0.8316 | F1 0.8330 | train 34s | peak 219 MB | skipped 0
scratch | seed 43 | 100% train (19923) | T = 256
  epoch 1: train loss 0.5747 | val loss 0.4339 | val acc 0.7934 | 6s
  epoch 2: train loss 0.3771 | val loss 0.3752 | val acc 0.8376 | 6s
  epoch 3: train loss 0.2915 | val loss 0.3580 | val acc 0.8426 | 6s
  epoch 4: train loss 0.2332 | val loss 0.3818 | val acc 0.8458 | 6s
  epoch 5: train loss 0.1858 | val loss 0.4171 | val acc 0.8450 | 6s
  epoch 6: train loss 0.1530 | val loss 0.4488 | val acc 0.8428 | 6s
  → test acc 0.8392 | F1 0.8336 | train 33s | peak 155 MB | skipped 0
scratch | seed 44 | 100% train (19923) | T = 256
  epoch 1: train loss 0.5807 | val loss 0.4345 | val acc 0.8020 | 6s
  epoch 2: train loss 0.3831 | val loss 0.3699 | val acc 0.8372 | 6s
  epoch 3: train loss 0.2912 | val loss 0.3667 | val acc 0.8478 | 6s
  epoch 4: train loss 0.2308 | val loss 0.3881 | val acc 0.8450 | 6s
  epoch 5: train loss 0.1876 | val loss 0.4105 | val acc 0.8430 | 6s
  epoch 6: train loss 0.1546 | val loss 0.4451 | val acc 0.8434 | 6s
  → test acc 0.8344 | F1 0.8389 | train 33s | peak 155 MB | skipped 0
pretrained | seed 42 | 100% train (19923) | T = 256
  epoch 1: train loss 0.3089 | val loss 0.2379 | val acc 0.9058 | 289s
  epoch 2: train loss 0.1652 | val loss 0.2792 | val acc 0.9103 | 289s
  → test acc 0.9104 | F1 0.9095 | train 484s | peak 1951 MB | skipped 0
pretrained | seed 43 | 100% train (19923) | T = 256
  epoch 1: train loss 0.3062 | val loss 0.2376 | val acc 0.9050 | 1678s
  epoch 2: train loss 0.1704 | val loss 0.2703 | val acc 0.9131 | 287s
  → test acc 0.9092 | F1 0.9110 | train 485s | peak 1964 MB | skipped 1
pretrained | seed 44 | 100% train (19923) | T = 256
  epoch 1: train loss 0.3137 | val loss 0.2858 | val acc 0.8882 | 292s
  epoch 2: train loss 0.1684 | val loss 0.2851 | val acc 0.9119 | 297s
  → test acc 0.9137 | F1 0.9138 | train 484s | peak 1954 MB | skipped 0
```

| model | accuracy | precision | recall | f1 | train_time | params | runs |
|---|---|---|---|---|---|---|---|
| **scratch** | 0.8350 ± 0.0038 | 0.8353 | 0.8362 | 0.8352 ± 0.0032 | 33.0790 | 4039554 | 3 |
| **pretrained** | 0.9111 ± 0.0023 | 0.9085 | 0.9146 | 0.9114 ± 0.0022 | 484.4285 | 66364418 | 3 |

```
fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
for r, c in [(scratch42, "C0"), (ft42, "C3")]:
    h = pd.DataFrame(r["hist"])
    ax[0].plot(h.epoch, h.train_loss, "--", c=c, label=f"{r['model']} train")
    ax[0].plot(h.epoch, h.val_loss, "-o", c=c, label=f"{r['model']} val")
    ax[1].plot(h.epoch, h.val_acc, "-o", c=c, label=r["model"])
ax[0].set(xlabel="epoch", ylabel="loss", title="Loss (seed 42)"); ax[0].legend(fontsize=8)
ax[1].set(xlabel="epoch", ylabel="validation accuracy", title="Validation accuracy (seed 42)");
ax[1].legend(fontsize=8)
plt.tight_layout(); plt.savefig("images/learning_curves.png", dpi=120); plt.show()
for r in (scratch42, ft42):
    print(r["model"], pd.DataFrame(r["hist"]).round(4).to_string(index=False), sep="\n")
```



```
scratch
 epoch  train_loss  val_loss  val_acc
     1      0.5641    0.4556   0.7802
     2      0.3876    0.3916   0.8191
     3      0.3024    0.3703   0.8388
     4      0.2432    0.3953   0.8384
     5      0.1968    0.4089   0.8374
     6      0.1631    0.4422   0.8372
pretrained
 epoch  train_loss  val_loss  val_acc
     1      0.3089    0.2379   0.9058
     2      0.1652    0.2792   0.9103
```

# 4. Exp 2 — Frozen encoder so với fine-tuning

Frozen: chạy DistilBERT pretrained (không cập nhật) **một lần** trên mọi review, lưu vector [CLS] và mean của token thật, rồi chỉ train head (dropout + Linear, 1 538 tham số) bằng Adam lr 1e-3, 30 epoch, chọn epoch theo validation loss. Thời gian train = thời gian trích feature của train + validation cộng thời gian train head.

```
encoder = pc.PretrainedClassifier().encoder
feats, ext_time = {}, {}
for k in ["train", "val", "test"]:
    t0 = time.time()
    feats[k] = pc.extract_features(encoder, ids[k], 256)
    ext_time[k] = time.time() - t0
print({k: f"{v:.0f}s" for k, v in ext_time.items()})

def run_frozen(pool, frac=1.0, seed=42):
    j = 0 if pool == "cls" else 1
    idx = subset(frac)
    t0 = time.time()
    head, hist = pc.train_head(feats["train"][j][idx], y["train"][idx], feats["val"][j], y["val"], seed=seed)
    t_head = time.time() - t0
    p = pc.head_proba(head, feats["test"][j])
    return {"model": f"frozen ({pool})", "seed": seed, "frac": frac, "T": 256, "n_train": len(idx),
            "params": sum(q.numel() for q in head.parameters()),
            "train_time": ext_time["train"] * frac + ext_time["val"] + t_head, "head_time": t_head,
            **pc.metrics(y["test"], p), "proba": p, "head": head, "hist": hist}

frozen = {pool: run_frozen(pool) for pool in ["cls", "mean"]}
exp2 = [frozen["cls"], frozen["mean"]] + [r for r in exp1 if r["model"] == "pretrained"]
tab2 = summary(exp2, "model")
tab2.insert(0, "encoder trainable?", ["No", "No", "Yes"])
tab2["trainable params"] = [1538, 1538, ft42["params"]]
for r in frozen.values():
    for c in ("accuracy", "precision", "recall", "f1", "train_time"):
        log("Exp2", r["model"], c, float(r[c]))
tab2
```

```
{'train': '223s', 'val': '56s', 'test': '281s'}
```

| model | encoder trainable? | accuracy | precision | recall | f1 | train_time | params | runs | trainable params |
|---|---|---|---|---|---|---|---|---|---|
| **frozen (cls)** | No | 0.8526 ± nan | 0.8671 | 0.8328 | 0.8496 ± nan | 290.0590 | 1538 | 1 | 1538 |
| **frozen (mean)** | No | 0.8586 ± nan | 0.8730 | 0.8394 | 0.8559 ± nan | 288.6074 | 1538 | 1 | 1538 |
| **pretrained** | Yes | 0.9111 ± 0.0023 | 0.9085 | 0.9146 | 0.9114 ± 0.0022 | 484.4285 | 66364418 | 3 | 66364418 |

# 5. Exp 3 — Lượng training data (10 / 25 / 50 / 100%)

Subset lấy ngẫu nhiên (seed 42) và lồng nhau. Validation và test giữ nguyên. Pretrained và from scratch chạy 1 seed cho 10–50%; điểm 100% là trung bình 3 seed của Exp 1. Frozen (CLS và mean) dùng lại feature đã cache.

```
exp3 = []
for frac in [0.1, 0.25, 0.5]:
    exp3.append(run("scratch", frac=frac))
    exp3.append(run("pretrained", frac=frac))
    for pool in ["cls", "mean"]:
        exp3.append(run_frozen(pool, frac=frac))
exp3 += [r for r in exp1] + [frozen["cls"], frozen["mean"]]
d3 = pd.DataFrame([{k: r[k] for k in ("model", "seed", "frac", "n_train", "accuracy", "f1", "train_time")} for r in
exp3])
tab3 = d3.groupby(["model", "frac"]).agg(n_train=("n_train", "first"), accuracy=("accuracy", "mean"),
                                         f1=("f1", "mean"), train_time=("train_time", "mean"), runs=("seed",
"size")).round(4)
for (m, f), r in tab3.iterrows():
    log("Exp3", f"{m} frac={f}", "accuracy", r.accuracy); log("Exp3", f"{m} frac={f}", "f1", r.f1)
tab3
```

```
scratch | seed 42 | 10% train (1992) | T = 256
  epoch 1: train loss 0.6915 | val loss 0.6648 | val acc 0.6270 | 1s
  epoch 2: train loss 0.6533 | val loss 0.7382 | val acc 0.5192 | 1s
  epoch 3: train loss 0.6160 | val loss 0.5985 | val acc 0.6792 | 1s
  epoch 4: train loss 0.5583 | val loss 0.5734 | val acc 0.7027 | 1s
  epoch 5: train loss 0.5065 | val loss 0.5580 | val acc 0.7141 | 1s
  epoch 6: train loss 0.4759 | val loss 0.5620 | val acc 0.7197 | 1s
  → test acc 0.7115 | F1 0.7076 | train 4s | peak 156 MB | skipped 0
pretrained | seed 42 | 10% train (1992) | T = 256
  epoch 1: train loss 0.4456 | val loss 0.2942 | val acc 0.8745 | 82s
  epoch 2: train loss 0.2132 | val loss 0.2943 | val acc 0.8836 | 97s
  → test acc 0.8763 | F1 0.8774 | train 49s | peak 1954 MB | skipped 0
scratch | seed 42 | 25% train (4981) | T = 256
  epoch 1: train loss 0.6725 | val loss 0.6060 | val acc 0.6764 | 2s
  epoch 2: train loss 0.5675 | val loss 0.5089 | val acc 0.7503 | 2s
  epoch 3: train loss 0.4426 | val loss 0.4696 | val acc 0.7776 | 2s
  epoch 4: train loss 0.3694 | val loss 0.4605 | val acc 0.7830 | 2s
  epoch 5: train loss 0.3132 | val loss 0.4693 | val acc 0.7912 | 2s
  epoch 6: train loss 0.2725 | val loss 0.4814 | val acc 0.7916 | 2s
  → test acc 0.7784 | F1 0.7639 | train 9s | peak 156 MB | skipped 0
pretrained | seed 42 | 25% train (4981) | T = 256
  epoch 1: train loss 0.3890 | val loss 0.2559 | val acc 0.8908 | 112s
  epoch 2: train loss 0.1903 | val loss 0.2863 | val acc 0.8944 | 119s
  → test acc 0.8942 | F1 0.8957 | train 122s | peak 1954 MB | skipped 0
scratch | seed 42 | 50% train (9962) | T = 256
  epoch 1: train loss 0.6271 | val loss 0.5308 | val acc 0.7344 | 4s
  epoch 2: train loss 0.4557 | val loss 0.4370 | val acc 0.7946 | 3s
  epoch 3: train loss 0.3645 | val loss 0.3990 | val acc 0.8173 | 3s
  epoch 4: train loss 0.2976 | val loss 0.4226 | val acc 0.8171 | 3s
  epoch 5: train loss 0.2467 | val loss 0.4372 | val acc 0.8191 | 4s
  epoch 6: train loss 0.2086 | val loss 0.4615 | val acc 0.8155 | 4s
  → test acc 0.8134 | F1 0.8169 | train 18s | peak 156 MB | skipped 0
pretrained | seed 42 | 50% train (9962) | T = 256
  epoch 1: train loss 0.3413 | val loss 0.2557 | val acc 0.8988 | 166s
  epoch 2: train loss 0.1724 | val loss 0.2874 | val acc 0.9038 | 182s
  → test acc 0.8973 | F1 0.9010 | train 243s | peak 1952 MB | skipped 0
```

| | | n_train | accuracy | f1 | train_time | runs |
|---|---|---|---|---|---|---|
| **model** | **frac** | | | | | |
| **frozen (cls)** | **0.10** | 1992 | 0.8367 | 0.8334 | 79.9553 | 1 |
| | **0.25** | 4981 | 0.8449 | 0.8403 | 115.3078 | 1 |
| | **0.50** | 9962 | 0.8502 | 0.8483 | 175.4187 | 1 |
| | **1.00** | 19923 | 0.8526 | 0.8496 | 290.0590 | 1 |
| **frozen (mean)** | **0.10** | 1992 | 0.8374 | 0.8339 | 79.9461 | 1 |
| | **0.25** | 4981 | 0.8486 | 0.8458 | 115.4894 | 1 |
| | **0.50** | 9962 | 0.8542 | 0.8527 | 173.9673 | 1 |
| | **1.00** | 19923 | 0.8586 | 0.8559 | 288.6074 | 1 |
| **pretrained** | **0.10** | 1992 | 0.8763 | 0.8774 | 49.1583 | 1 |
| | **0.25** | 4981 | 0.8942 | 0.8957 | 121.5893 | 1 |
| | **0.50** | 9962 | 0.8973 | 0.9010 | 242.5616 | 1 |
| | **1.00** | 19923 | 0.9111 | 0.9114 | 484.4285 | 3 |
| **scratch** | **0.10** | 1992 | 0.7115 | 0.7076 | 3.6397 | 1 |
| | **0.25** | 4981 | 0.7784 | 0.7639 | 8.9087 | 1 |
| | **0.50** | 9962 | 0.8134 | 0.8169 | 17.9112 | 1 |
| | **1.00** | 19923 | 0.8350 | 0.8352 | 33.0790 | 3 |

```
acc3 = tab3.accuracy.unstack(0)
print("Accuracy theo % data:"); print(acc3.round(4))
print("\nGiảm khi đi từ 100% xuống 10% (điểm accuracy):")
print((acc3.loc[1.0] - acc3.loc[0.1]).round(4))
print("\nKhoảng cách pretrained − scratch:")
print((acc3["pretrained"] - acc3["scratch"]).round(4))
```

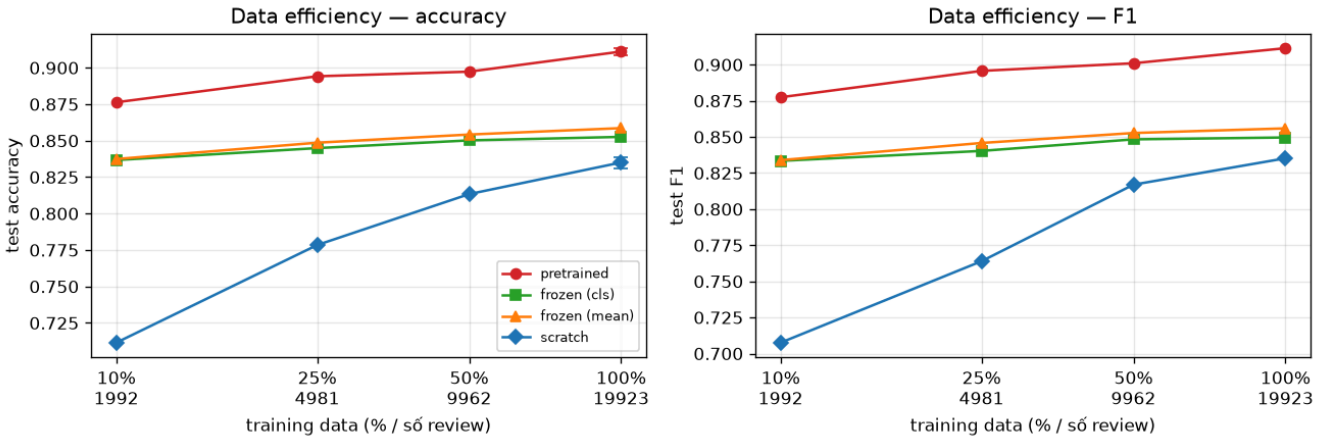
```
Accuracy theo % data:
model  frozen (cls)  frozen (mean)  pretrained  scratch
frac
0.10         0.8367         0.8374      0.8763   0.7115
0.25         0.8449         0.8486      0.8942   0.7784
0.50         0.8502         0.8542      0.8973   0.8134
1.00         0.8526         0.8586      0.9111   0.8350

Giảm khi đi từ 100% xuống 10% (điểm accuracy):
model
frozen (cls)     0.0159
frozen (mean)    0.0212
pretrained       0.0348
scratch          0.1235
dtype: float64

Khoảng cách pretrained − scratch:
frac
0.10    0.1648
0.25    0.1158
0.50    0.0839
1.00    0.0761
dtype: float64
```

```
fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
style = {"pretrained": ("C3", "o"), "frozen (cls)": ("C2", "s"), "frozen (mean)": ("C1", "^"), "scratch": ("C0",
"D")}
for m, (c, mk) in style.items():
    t = tab3.loc[m]
    x = t.n_train.values
    ax[0].plot(x, t.accuracy.values, marker=mk, c=c, label=m)
    ax[1].plot(x, t.f1.values, marker=mk, c=c, label=m)
    if m in ("pretrained", "scratch"):                    # độ lệch giữa 3 seed ở 100%
        v = d3[(d3.model == m) & (d3.frac == 1.0)].accuracy
```

```
        ax[0].errorbar(x[-1], v.mean(), yerr=v.std(), c=c, capsize=4)
for a, name in zip(ax, ["accuracy", "F1"]):
    a.set_xscale("log"); a.xaxis.set_minor_locator(NullLocator()); a.set_xticks(tab3.loc["scratch"].n_train.values)
    a.set_xticklabels([f"{f:.0%}\n{n}" for f, n in zip([0.1, 0.25, 0.5, 1.0], tab3.loc["scratch"].n_train.values)])
    a.set(xlabel="training data (% / số review)", ylabel=f"test {name}", title=f"Data efficiency — {name}")
    a.grid(alpha=0.3)
ax[0].legend(fontsize=8)
plt.tight_layout(); plt.savefig("images/data_efficiency.png", dpi=120); plt.show()
```




# 6. Exp 4 — Max sequence length 64 / 128 / 256

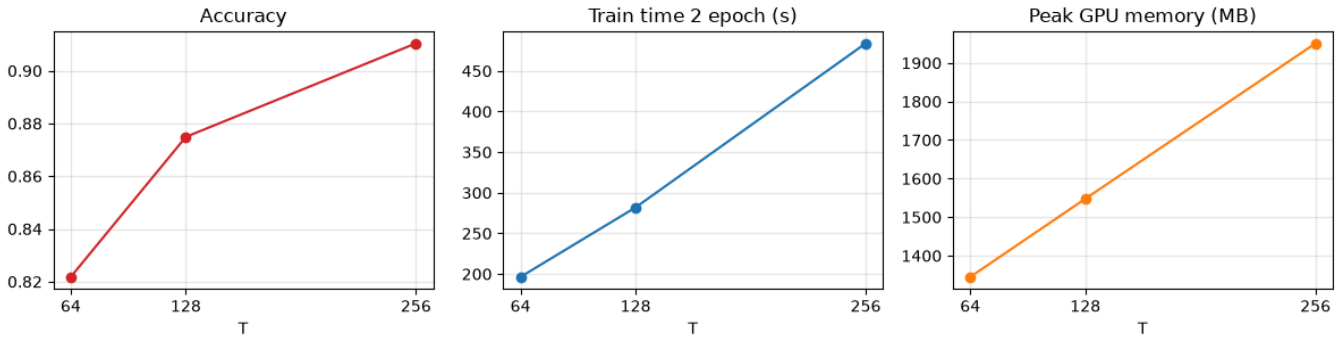
Pretrained fine-tuning, seed 42, 100% data. Điểm T = 256 lấy từ Exp 1 (seed 42). Bộ nhớ là đỉnh `torch.cuda.max_memory_allocated` trong lúc train (gồm trọng số, optimizer state, activation).

```
exp4 = [run("pretrained", T=T) for T in [64, 128]] + [ft42]
tab4 = pd.DataFrame([{"max length": r["T"], "accuracy": r["accuracy"], "f1": r["f1"], "train time (s)":
r["train_time"],
                      "test time (s)": r["test_time"], "peak GPU mem (MB)": r["peak_mem_mb"],
                      "% review bị cắt": np.mean(n_tok > r["T"])} for r in exp4]).set_index("max length").round(4)
for T, r in tab4.iterrows():
    for c in tab4.columns:
        log("Exp4", f"T={T}", c, float(r[c]))
tab4
```

```
pretrained | seed 42 | 100% train (19923) | T = 64
  epoch 1: train loss 0.4559 | val loss 0.3741 | val acc 0.8285 | 102s
  epoch 2: train loss 0.2957 | val loss 0.3982 | val acc 0.8358 | 101s
  → test acc 0.8218 | F1 0.8206 | train 196s | peak 1345 MB | skipped 21
pretrained | seed 42 | 100% train (19923) | T = 128
  epoch 1: train loss 0.3735 | val loss 0.2892 | val acc 0.8745 | 148s
  epoch 2: train loss 0.2180 | val loss 0.3254 | val acc 0.8773 | 148s
  → test acc 0.8749 | F1 0.8738 | train 282s | peak 1548 MB | skipped 63
```

| max length | accuracy | f1 | train time (s) | test time (s) | peak GPU mem (MB) | % review bị cắt |
|---|---|---|---|---|---|---|
| **64** | 0.8218 | 0.8206 | 196.3891 | 18.6463 | 1344.9800 | 0.9788 |
| **128** | 0.8749 | 0.8738 | 281.9496 | 34.6844 | 1547.6909 | 0.8781 |
| **256** | 0.9104 | 0.9095 | 483.9585 | 232.4283 | 1950.7378 | 0.4120 |

```
fig, ax = plt.subplots(1, 3, figsize=(12, 3.2))
ax[0].plot(tab4.index, tab4.accuracy, "-o", c="C3"); ax[0].set(title="Accuracy", xlabel="T")
ax[1].plot(tab4.index, tab4["train time (s)"], "-o", c="C0"); ax[1].set(title="Train time 2 epoch (s)", xlabel="T")
ax[2].plot(tab4.index, tab4["peak GPU mem (MB)"], "-o", c="C1"); ax[2].set(title="Peak GPU memory (MB)",
xlabel="T")
for a in ax:
    a.set_xticks(tab4.index); a.grid(alpha=0.3)
plt.tight_layout(); plt.savefig("images/sequence_length.png", dpi=120); plt.show()
```




# 7. Exp 5 — Domain shift

- **Domain A (in-domain):** IMDb, review phim.
- **Domain B (out-of-domain):** Yelp (review nhà hàng / dịch vụ) và Amazon (review sản phẩm). Mỗi bộ lấy 2 000 mẫu cân bằng nhãn từ tập test gốc.

Không train lại: dùng đúng 3 model đã train trên IMDb (seed 42), T = 256.

```
dom = {name: pc.load_domain(name) for name in ["yelp", "amazon"]}
for name, d in dom.items():
    d_ids = pc.encode(tokenizer, d.text)
    ids[name], y[name] = d_ids, d.label.values
    print(f"{name}: {len(d)} mẫu, median {np.median([len(x) for x in d_ids]):.0f} token | ví dụ: {d.text[0]
[:150]}…")
    feats[name] = pc.extract_features(encoder, d_ids, 256)

rows5 = []
for name in ["test", "yelp", "amazon"]:
    preds = {"scratch": pc.predict_proba(scratch42["model_obj"], ids[name], 256),
             "frozen (cls)": pc.head_proba(frozen["cls"]["head"], feats[name][0]),
             "frozen (mean)": pc.head_proba(frozen["mean"]["head"], feats[name][1]),
             "pretrained": pc.predict_proba(ft42["model_obj"], ids[name], 256)}
    for m, p in preds.items():
        rows5.append({"domain": "IMDb (in-domain)" if name == "test" else name, "model": m, **pc.metrics(y[name],
p),
                      "mean confidence": np.maximum(p, 1 - p).mean()})
tab5 = pd.DataFrame(rows5).set_index(["model", "domain"]).sort_index().round(4)
for (m, d), r in tab5.iterrows():
    for c in tab5.columns:
        log("Exp5", f"{m} | {d}", c, float(r[c]))
tab5
```

```
yelp: 2000 mẫu, median 125 token | ví dụ: They say what you see is what you get and after seeing her amazing hair
and makeup I knew I had to have her do my makeup/hair for my wedding. She was …
amazon: 2000 mẫu, median 94 token | ví dụ: One happy camper!. Let's see...$5 cheaper than the local retailer AND
free shipping...how can you go wrong??? On top of all that, the optional "ball h…
```

| model | domain | accuracy | precision | recall | f1 | mean confidence |
|---|---|---|---|---|---|---|
| **frozen (cls)** | **IMDb (in-domain)** | 0.8526 | 0.8671 | 0.8328 | 0.8496 | 0.8488 |
| | **amazon** | 0.8735 | 0.8514 | 0.9050 | 0.8774 | 0.8787 |
| | **yelp** | 0.8750 | 0.8447 | 0.9190 | 0.8803 | 0.8698 |
| **frozen (mean)** | **IMDb (in-domain)** | 0.8586 | 0.8730 | 0.8394 | 0.8559 | 0.8495 |
| | **amazon** | 0.8595 | 0.8207 | 0.9200 | 0.8675 | 0.8754 |
| | **yelp** | 0.8570 | 0.8046 | 0.9430 | 0.8683 | 0.8608 |
| **pretrained** | **IMDb (in-domain)** | 0.9104 | 0.9191 | 0.9001 | 0.9095 | 0.9246 |
| | **amazon** | 0.9265 | 0.9169 | 0.9380 | 0.9273 | 0.9224 |
| | **yelp** | 0.9235 | 0.9206 | 0.9270 | 0.9238 | 0.9022 |
| **scratch** | **IMDb (in-domain)** | 0.8316 | 0.8259 | 0.8403 | 0.8330 | 0.8736 |
| | **amazon** | 0.7845 | 0.7565 | 0.8390 | 0.7956 | 0.8797 |
| | **yelp** | 0.7960 | 0.7741 | 0.8360 | 0.8038 | 0.8608 |

```
acc5 = tab5.accuracy.unstack()
print("Accuracy:"); print(acc5)
print("\nOut-of-domain − in-domain:")
print(acc5[["amazon", "yelp"]].sub(acc5["IMDb (in-domain)"], axis=0).round(4))
```

```
Accuracy:
domain         IMDb (in-domain)  amazon    yelp
model
frozen (cls)             0.8526  0.8735  0.8750
frozen (mean)            0.8586  0.8595  0.8570
pretrained               0.9104  0.9265  0.9235
scratch                  0.8316  0.7845  0.7960

Out-of-domain − in-domain:
domain         amazon    yelp
model
frozen (cls)   0.0209  0.0224
frozen (mean)  0.0009 -0.0016
pretrained     0.0161  0.0131
scratch       -0.0471 -0.0356
```

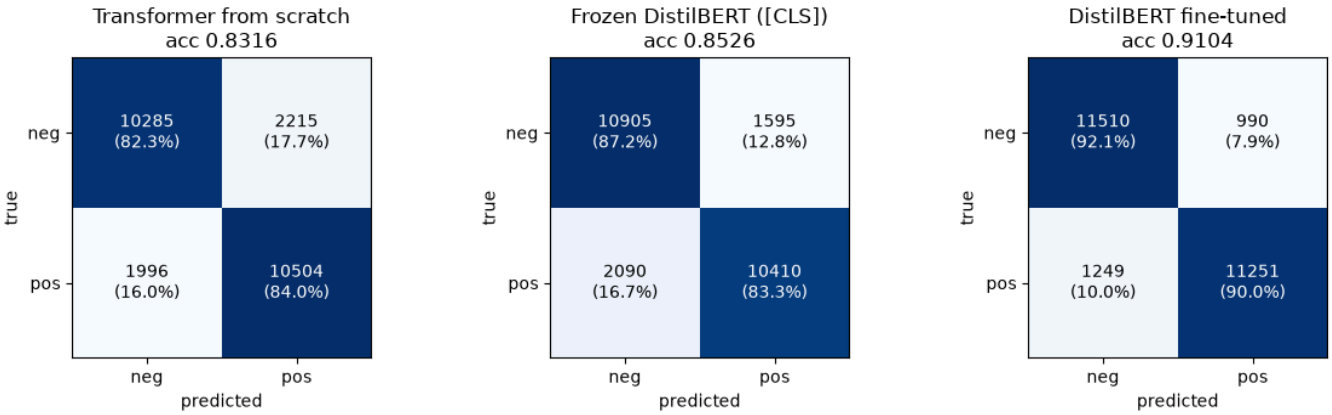
# 8. Application — sentiment classification (pretrained fine-tuned, seed 42)

```
p_ft, y_te = ft42["proba"], y["test"]
pred_ft = (p_ft > 0.5).astype(int)
app = pd.DataFrame({m: pc.metrics(y_te, p) for m, p in
                    [("scratch", scratch42["proba"]), ("frozen (cls)", frozen["cls"]["proba"]),
                     ("frozen (mean)", frozen["mean"]["proba"]), ("pretrained", p_ft)]}).T.round(4)
app
```

| | accuracy | precision | recall | f1 |
|---|---|---|---|---|
| **scratch** | 0.8316 | 0.8259 | 0.8403 | 0.8330 |
| **frozen (cls)** | 0.8526 | 0.8671 | 0.8328 | 0.8496 |
| **frozen (mean)** | 0.8586 | 0.8730 | 0.8394 | 0.8559 |
| **pretrained** | 0.9104 | 0.9191 | 0.9001 | 0.9095 |

```
fig, ax = plt.subplots(1, 3, figsize=(12, 3.6))
for a, (m, p) in zip(ax, [("Transformer from scratch", scratch42["proba"]), ("Frozen DistilBERT ([CLS])",
frozen["cls"]["proba"]),
```

```
                          ("DistilBERT fine-tuned", p_ft)]):
    cm = confusion_matrix(y_te, p > 0.5)
    a.imshow(cm, cmap="Blues")
    for i in range(2):
        for j in range(2):
            a.text(j, i, f"{cm[i, j]}\n({cm[i, j] / cm[i].sum():.1%})", ha="center", va="center",
                   color="white" if cm[i, j] > cm.max() / 2 else "black")
    a.set(xticks=[0, 1], yticks=[0, 1], xticklabels=["neg", "pos"], yticklabels=["neg", "pos"],
          xlabel="predicted", ylabel="true", title=f"{m}\nacc {np.mean((p > 0.5) == y_te):.4f}")
plt.tight_layout(); plt.savefig("images/confusion_matrix.png", dpi=120); plt.show()
cm = confusion_matrix(y_te, pred_ft)
print("Fine-tuned: TN, FP, FN, TP =", cm.ravel())
for k, v in zip(["TN", "FP", "FN", "TP"], cm.ravel()):
    log("Application", "pretrained seed=42", k, int(v))
```




```
Fine-tuned: TN, FP, FN, TP = [11510   990  1249 11251]
```

```
own = ["This movie was absolutely wonderful, I loved every minute.",
       "Not bad at all, actually one of the better films this year.",
       "I expected to hate it, but it turned out to be great.",
       "It was great... if you enjoy falling asleep.",
       "The acting was fine but the plot made no sense whatsoever.",
       "A masterpiece of boredom.",
       "Oh great, another two hours of my life I will never get back.",
       "The food was cold and the waiter was rude.",
       "Battery life is excellent and it charges quickly.",
       "Phim rất hay, diễn viên đóng tuyệt vời."]
own_ids = pc.encode(tokenizer, own)
p_own = {"scratch": pc.predict_proba(scratch42["model_obj"], own_ids, 256),
         "pretrained": pc.predict_proba(ft42["model_obj"], own_ids, 256)}
pd.set_option("display.max_colwidth", 80)
pd.DataFrame({"text": own, "P(pos) scratch": p_own["scratch"].round(3), "P(pos) fine-tuned":
p_own["pretrained"].round(3)})
```

| | text | P(pos) scratch | P(pos) fine-tuned |
|---|---|---|---|
| **0** | This movie was absolutely wonderful, I loved every minute. | 1.000 | 0.996 |
| **1** | Not bad at all, actually one of the better films this year. | 0.021 | 0.942 |
| **2** | I expected to hate it, but it turned out to be great. | 0.998 | 0.940 |
| **3** | It was great... if you enjoy falling asleep. | 0.996 | 0.973 |
| **4** | The acting was fine but the plot made no sense whatsoever. | 0.001 | 0.009 |
| **5** | A masterpiece of boredom. | 0.995 | 0.853 |
| **6** | Oh great, another two hours of my life I will never get back. | 0.999 | 0.712 |
| **7** | The food was cold and the waiter was rude. | 0.237 | 0.045 |
| **8** | Battery life is excellent and it charges quickly. | 1.000 | 0.943 |
| **9** | Phim rất hay, diễn viên đóng tuyệt vời. | 0.966 | 0.647 |

# 9. Confidence analysis

`confidence = max(softmax(logits))`. Chia ba nhóm: **High** ≥ 0.95, **Medium** 0.75–0.95, **Low** < 0.75.

```
def conf_table(p, yy):
    conf = np.maximum(p, 1 - p)
    correct = (p > 0.5) == yy
    grp = pd.cut(conf, [0.5, 0.75, 0.95, 1.0001], labels=["Low (<0.75)", "Medium (0.75–0.95)", "High (≥0.95)"],
include_lowest=True)
    t = pd.DataFrame({"group": grp, "correct": correct, "conf": conf}).groupby("group", observed=False).agg(
        n=("correct", "size"), accuracy=("correct", "mean"), mean_conf=("conf", "mean"), errors=("correct", lambda
c: int((~c).sum())))
    t["% mẫu"] = t.n / t.n.sum()
    t["% lỗi"] = t.errors / t.errors.sum()
    return t.round(4)

def ece(p, yy, bins=10):
    conf = np.maximum(p, 1 - p); correct = (p > 0.5) == yy
    edges = np.linspace(0.5, 1, bins + 1); e = 0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi) if lo > 0.5 else (conf >= lo) & (conf <= hi)
        if m.any():
            e += m.mean() * abs(correct[m].mean() - conf[m].mean())
    return e

for name, p in [("pretrained fine-tuned", p_ft), ("scratch", scratch42["proba"]), ("frozen (cls)", frozen["cls"]
["proba"])]:
    t = conf_table(p, y_te)
    print(f"\n{name}: ECE = {ece(p, y_te):.4f}"); print(t)
    log("Confidence", name, "ECE", ece(p, y_te))
    for g, r in t.iterrows():
        log("Confidence", f"{name} | {g}", "accuracy", r.accuracy); log("Confidence", f"{name} | {g}", "n",
int(r.n))
wrong = (pred_ft != y_te)
conf_ft = np.maximum(p_ft, 1 - p_ft)
print(f"\nFine-tuned: {wrong.sum()} lỗi; {np.mean(conf_ft[wrong] > 0.95):.1%} lỗi có confidence > 0.95, "
      f"{np.mean(conf_ft[wrong] > 0.99):.1%} lỗi có confidence > 0.99")
```

```
pretrained fine-tuned: ECE = 0.0142
                       n  accuracy  mean_conf  errors   % mẫu   % lỗi
group
Low (<0.75)          2605    0.6138     0.6346    1006  0.1042  0.4493
Medium (0.75–0.95)   5538    0.8346     0.8781     916  0.2215  0.4091
High (≥0.95)        16857    0.9812     0.9847     317  0.6743  0.1416

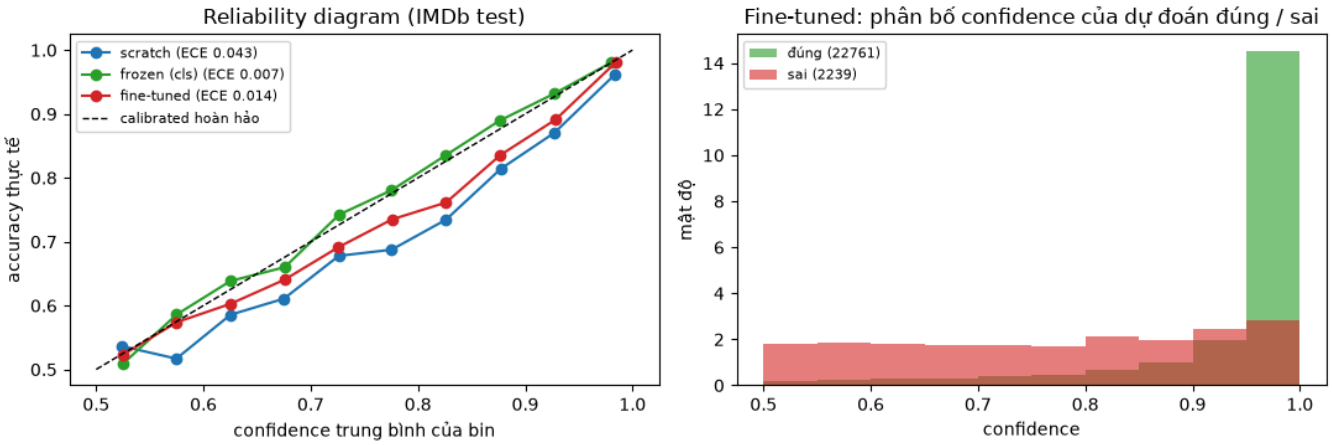
scratch: ECE = 0.0429
                       n  accuracy  mean_conf  errors   % mẫu   % lỗi
group
Low (<0.75)          5131    0.5913     0.6323    2097  0.2052  0.4980
Medium (0.75–0.95)   8421    0.8011     0.8711    1675  0.3368  0.3978
High (≥0.95)        11448    0.9617     0.9835     439  0.4579  0.1043

frozen (cls): ECE = 0.0068
                       n  accuracy  mean_conf  errors   % mẫu   % lỗi
group
Low (<0.75)          6202    0.6324     0.6304    2280  0.2481  0.6187
Medium (0.75–0.95)  10038    0.8768     0.8688    1237  0.4015  0.3357
High (≥0.95)         8760    0.9808     0.9806     168  0.3504  0.0456

Fine-tuned: 2239 lỗi; 14.2% lỗi có confidence > 0.95, 1.2% lỗi có confidence > 0.99
```

```
fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
bins = np.linspace(0.5, 1, 11)
for name, p, c in [("scratch", scratch42["proba"], "C0"), ("frozen (cls)", frozen["cls"]["proba"], "C2"), ("fine-
tuned", p_ft, "C3")]:
    conf = np.maximum(p, 1 - p); correct = (p > 0.5) == y_te
    idx = np.clip(np.digitize(conf, bins) - 1, 0, 9)
    xs = [conf[idx == b].mean() for b in range(10) if (idx == b).sum() > 20]
    ys = [correct[idx == b].mean() for b in range(10) if (idx == b).sum() > 20]
    ax[0].plot(xs, ys, "-o", c=c, label=f"{name} (ECE {ece(p, y_te):.3f})")
ax[0].plot([0.5, 1], [0.5, 1], "k--", lw=1, label="calibrated hoàn hảo")
ax[0].set(xlabel="confidence trung bình của bin", ylabel="accuracy thực tế", title="Reliability diagram (IMDb
test)"); ax[0].legend(fontsize=8)
ax[1].hist(conf_ft[~wrong], bins=bins, alpha=0.6, label=f"đúng ({(~wrong).sum()})", color="C2", density=True)
ax[1].hist(conf_ft[wrong], bins=bins, alpha=0.6, label=f"sai ({wrong.sum()})", color="C3", density=True)
ax[1].set(xlabel="confidence", ylabel="mật độ", title="Fine-tuned: phân bố confidence của dự đoán đúng / sai");
```

```
ax[1].legend(fontsize=8)
plt.tight_layout(); plt.savefig("images/confidence_analysis.png", dpi=120); plt.show()
```




# 10. Error analysis (fine-tuned, seed 42)

```
te = data["test"].assign(p=p_ft, pred=pred_ft, conf=conf_ft, n_tok=[len(x) for x in ids["test"]])
te["truncated"] = te.n_tok > 256
err = te[te.pred != te.label]
print(f"Lỗi: {len(err)} (FP {((err.pred == 1)).sum()}, FN {((err.pred == 0)).sum()})")
print(f"Tỉ lệ lỗi: review bị cắt ở 256 token {1 - (te[te.truncated].pred == te[te.truncated].label).mean():.4f}, "
      f"không bị cắt {1 - (te[~te.truncated].pred == te[~te.truncated].label).mean():.4f}")
te["len_bin"] = pd.qcut(te.n_tok, 4, labels=["Q1 ngắn", "Q2", "Q3", "Q4 dài"])
print(te.groupby("len_bin", observed=False).apply(lambda g: pd.Series({"token median": g.n_tok.median(),
"accuracy": (g.pred == g.label).mean()})).round(4))

def show(df, title):
    print(f"\n{'=' * 20} {title}")
    for i, r in df.iterrows():
        print(f"\n[#{i}] true {r.label} | pred {r.pred} | P(pos) {r.p:.4f} | conf {r.conf:.4f} | {r.n_tok} token{'
(bị cắt)' if r.truncated else ''}")
        print(r.text[:900] + ("…" if len(r.text) > 900 else ""))

rng = np.random.default_rng(1)
fp, fn = err[err.pred == 1], err[err.pred == 0]
show(fp.sample(4, random_state=1), "FALSE POSITIVES (ngẫu nhiên)")
show(fn.sample(4, random_state=1), "FALSE NEGATIVES (ngẫu nhiên)")
show(err.sort_values("conf", ascending=False).head(4), "SAI NHƯNG RẤT TỰ TIN")
```

Lỗi: 2239 (FP 990, FN 1249)
Tỉ lệ lỗi: review bị cắt ở 256 token 0.1237, không bị cắt 0.0667

| len_bin | token median | accuracy |
|---|---|---|
| Q1 ngắn | 128.0 | 0.9309 |
| Q2 | 184.0 | 0.9372 |
| Q3 | 269.0 | 0.9166 |
| Q4 dài | 509.5 | 0.8570 |

========== FALSE POSITIVES (ngẫu nhiên)

[#4116] true 0 | pred 1 | P(pos) 0.6553 | conf 0.6553 | 178 token
I bought this Chuck Norris DVD knowing that it was one of his earliest films, and it shows. We all know that he will never win an Oscar for his acting, but that's not what we watch him for. Although there have been a few earthquakes in California since this movie was made, there never was any desert or hills between Hwy.99 and I 5. Billy was supposedly crossing over from 99 to 5 along 120, a distance of less that 15 miles. I wish that the writers, producers and directors of these movies would, at least, look at a map. As a truck driver who spends a lot of time in California, I could tell right from the start that the geography was wrong. However, there are worse ways to spend an hour and a half. So grab your Doritos and an adult beverage and enjoy a trip back in time.

[#10188] true 0 | pred 1 | P(pos) 0.8891 | conf 0.8891 | 512 token (bị cắt)
John Wayne is without a doubt one of the most popular and loved actors of all time. His career stretched over forty years, and within that time he starred in films such as "Angel and the Badman", "The Green Berets", "Sands of Iwo Jima", "Rio Bravo", "North to Alaska", and "The Undefeated". The film's listed above are hailed as some of his best, unlike this 1934 effort "Randy Rides Alone", which has been pretty much forgotten about as time's gone on, which is unsurprising, as it's nothing memorable apart from its very short running time of just 53 minutes. A young John Wayne plays Randy Bowers, who for reasons never really explained, arrives at a saloon in the middle of nowhere and finds that everyone inside has been killed. While looking around, a posse arrives and finds Randy there and they arrest him, accusing him of being a gang member and demand to know where the rest of his gang is.…

[#11985] true 0 | pred 1 | P(pos) 0.6564 | conf 0.6564 | 474 token (bị cắt)
Go to the video store and get the original. I do not understand why Hollywood has that need to take a perfect foreign movie and remake it. "Mostly Martha" or "Bella Martha" has a much better cast. Beginning with the heroine Martina Gedeck, who convinced me much more in the role of the work-obsessed perfectionist than the more famous Catherine Zeta Jones, to the Italian cook and the niece suddenly deprived of her mother and forced to live with an aunt, not fit for child-rearing. In many ways, the American version of the movie is a copy of the German original. They just exchanged the actors. However, they also changed the story because it would have been difficult and not very believable to materialize a father for the little girl in an American context. I was thinking about that. Maybe the father could have been Puerto Rican, or Cuban, or Mexican. Well, there are so many "guest workers" i…

[#8793] true 0 | pred 1 | P(pos) 0.8693 | conf 0.8693 | 185 token
Directed by Diane Keaton and adapted from a book by Franz Lidz. A young mother Selma Lidz(Andie MacDowell)is battling a very serious illness and her self proclaimed inventor husband Sid(John Tururro)is a little lacking in the emotions department. Unhappy with the new home situation, their sensitive son Steven(Nathan Watt)decides he wants to stay with his two eccentric uncles Danny(Michael Richards)and Melvin(Lou Cutell)until his mom is well. Steven seems to be happier and even takes interest in his strange uncle's living habits; he even decides he wants to change his name to Franz. Set in the early 60's, this drama is a bit comedic...change that to zany. Not being a MacDowell fan, UNSTRUNG HEROES assures my attitude; albeit I enjoyed the film and it is not a total waste.

========== FALSE NEGATIVES (ngẫu nhiên)

[#16685] true 1 | pred 0 | P(pos) 0.0233 | conf 0.9767 | 215 token
I've got to say it. Gary Busey saved this film. If it were not for his fine acting talents this film would have been sub par. I recommend the film but just barely. The biggest difficulty with the film is its broad disregard for historical, geographical and physical accuracy. For example, why is Lubbock green and hilly? How come Buddy's producer Petty ignored? Where are the daily trips to the Clovis, New Mexico studio? Why is Nashville treated as a racist hate camp out to destroy the Holly sound? Why was Buddy's two week courtship to Maria treated as a complex, taboo, race mixing stereotype? Why are Buddy's tile boxes as light as feathers? Finally, why are the Crickets portrayed as trouble making roadblocks to Holly's talent? Taken on their face these inaccuracies should spell doom for any film proclaiming itself, "The Buddy Holly Story." However, Busey does deliver a stirring portrayal a…

[#19636] true 1 | pred 0 | P(pos) 0.4569 | conf 0.5431 | 423 token (bị cắt)
Excuse me if I'm wrong, but "Cronica para un desayuno" could be one of the most gretatest films in tne mexican movie history for a lot of reasons. We can get a lot of ingredients for a perfect mexican middle-class family breakfast: a cup of Buñuel's surrealistic motifs with a little drop of Ripstenian desolated scenarios; a pound of phallus symbols around the film and a difficult psychological complexity, more than Todd Solondz's Happiness and the Dogma movement films. There`s a lot of sordidness, black humor, repressed dreams (Teodoro's dream of fly), incestual lack of control (the relation between Marcos and Luzma), no-sense parallel stories, a discrete violence, anachronisms and a little sign of hope (spotlighted in Luzma, the husband's home return and "Un poco Mas" used for leit-motif). Marcos (Bruno Bichir) is a charming character ( I think in a Bukoskian way), the king-without a-cr…

[#23886] true 1 | pred 0 | P(pos) 0.4620 | conf 0.5380 | 141 token
if you are dating a girl that is into wicca! many parts of this movie were killer and the feeling you get from this movie while watching it and immediately after is enough alone to sit through it all but some things really bothered me about this film that i cant really put my finger on... i definitely think that this movie is one that any art

```
student or photography student should see b/c of the camera style and graininess that ive never seen but that is just it...it looks like an art student made it and the plot if there is any is so jumpy that you really need to be on a lot of drugs to get just what is going on

[#19513] true 1 | pred 0 | P(pos) 0.1852 | conf 0.8148 | 471 token (bị cắt)
Do-It-Yourself indie horror auteur Todd Sheets returns with another entertainingly atrocious nickel'n'dime shot-on-video clunker that's basically just a feeble excuse to sling around a lot of watery blood and gleaming guts as often as possible. An evil demonic scarecrow resurrects the dead as ravenous rot-faced zombies so they can feast on the living. A bunch of bickering college kids, a trio of dangerous escaped convicts led by the vicious Slade (Byron Nichodemus hamming it up to an outrageous degree), two equally savage sleazeball hoodlums, and a trio of hottie sisters all have to do their best to survive this harrowing ordeal. That's it for the needlessly muddled and convoluted plot, but fortunately what this hilariously horrendous hoot lacks in narrative coherence (plenty) it more than compensates for with a pleasing plethora of gloriously gross'n'graphic gore. Disgusting highlights …

==================== SAI NHƯNG RẤT TỰ TIN

[#12061] true 0 | pred 1 | P(pos) 0.9964 | conf 0.9964 | 100 token
This movie was pure genius. John Waters is brilliant. It is hilarious and I am not sick of it even after seeing it about 20 times since I bought it a few months ago. The acting is great, although Ricki Lake could have been better. And Johnny Depp is magnificent. He is such a beautiful man and a very talented actor. And seeing most of Johnny's movies, this is probably my favorite. I give it 9.5/10. Rent it today!

[#4118] true 0 | pred 1 | P(pos) 0.9959 | conf 0.9959 | 73 token
This is definitely one of the best Kung fu movies in the history of Cinema. The screenplay is really well done (which is not often the case for this type of movies) and you can see that Chuck (in one of his first role)is a great actor. The final fight with the sherif deputy in the bullring is a masterpiece!

[#2688] true 0 | pred 1 | P(pos) 0.9949 | conf 0.9949 | 54 token
I really liked this quirky movie. The characters are not the bland beautiful people that show up in so many movies and on TV. It has a realistic edge, with a captivating story line. The main title sequence alone makes this movie fun to watch.

[#19650] true 1 | pred 0 | P(pos) 0.0051 | conf 0.9949 | 512 token (bị cắt)
I just saw this for the first time in 10 or 15 years...maybe close to 20. In some ways, it was better than I remembered...in other, it was MUCH worse. First of all, there's the music. It's just plain awful. There are only 5 songs in the movie, most of them used more than once. The opening song is shrieked by a chorus of annoying children, and the disco-y title track is performed by Rick Dees. It doesn't get any worse than that. Even the background music is terrible, with much of it repeating the themes of the other nauseating tunes. We also get some truly lame slapstick, mostly in the opening credits. On the other hand, Bill Murray is spot on brilliant as usual...you have to wonder if he ad-libbed the whole thing, or if the writers just gave him all the funny lines. Or maybe he's just that great- turning a weak script into comic genius. The best part are his surreal PA announcements. ("L…
```

Đối chiếu: các lỗi của model fine-tuned có phải cũng là lỗi của model from scratch không?

```
pred_sc = scratch42["proba"] > 0.5
both = pd.crosstab(pd.Series(pred_ft == y_te, name="fine-tuned đúng"), pd.Series(pred_sc == y_te, name="scratch đúng"))
print(both)
print(f"\nTrong {wrong.sum()} lỗi của fine-tuned, scratch cũng sai {np.mean(pred_sc[wrong] != y_te[wrong]):.1%}")
```

```
scratch đúng     False  True
fine-tuned đúng
False             1295    944
True              2916  19845

Trong 2239 lỗi của fine-tuned, scratch cũng sai 57.8%
```

# 11. Lưu kết quả

```
pd.DataFrame(RESULTS).to_csv("results.csv", index=False)
print(len(RESULTS), "dòng → results.csv | tổng thời gian notebook:", f"{(time.time() - t_all) / 60:.1f} phút")
print("skipped steps (NaN) tổng:", int(sum(r["value"] for r in RESULTS if r["metric"] == "skipped_steps")),
      "| GPU retries:", sum(1 for r in RESULTS if r["experiment"] == "GPU retries"))
```

```
285 dòng → results.csv | tổng thời gian notebook: 114.3 phút
skipped steps (NaN) tổng: 85 | GPU retries: 0
```

# Error analysis — LAB 08

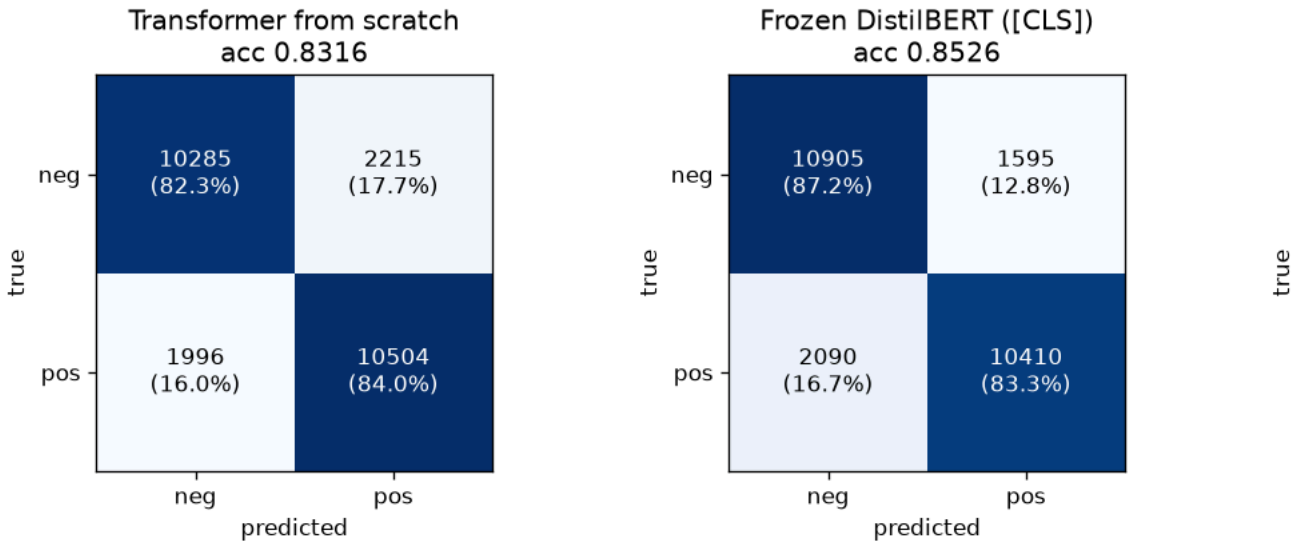
Model: DistilBERT fine-tuned (seed 42, T = 256, chọn epoch 1 theo validation loss). Đánh giá trên IMDb test (25 000 review). Số `#` là chỉ số dòng trong tập test.

## 1. Kết quả và confusion matrix

| Model | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|
| Transformer from scratch | 0.8316 | 0.8259 | 0.8403 | 0.8330 |
| Frozen DistilBERT ([CLS]) | 0.8526 | 0.8671 | 0.8328 | 0.8496 |
| Frozen DistilBERT (mean) | 0.8586 | 0.8730 | 0.8394 | 0.8559 |
| **DistilBERT fine-tuned** | **0.9104** | **0.9191** | 0.9001 | **0.9095** |

Confusion matrix của model fine-tuned:

| | pred neg | pred pos |
|---|---|---|
| **true neg** | 11 510 (TN) | 990 (FP) |
| **true pos** | 1 249 (FN) | 11 251 (TP) |




- **Có 2 239 lỗi; FN nhiều hơn FP (1 249 so với 990).** Model hơi thiên về đoán negative: precision 0.919 > recall 0.900. Review khen nhưng có đoạn chê dài dễ bị đoán negative (xem mục 3).
- **Lỗi phụ thuộc rõ vào độ dài:**
- review dài hơn 256 token (41% tập test) bị cắt và có tỉ lệ lỗi 12.4%, gấp đôi review không bị cắt (6.7%);
- theo quartile độ dài, accuracy lần lượt là 0.931 / 0.937 / 0.917 / **0.857** (quartile dài nhất, median 510 token).
- **Lỗi của hai model trùng một phần:** trong 2 239 lỗi của model fine-tuned, model from scratch cũng sai 57.8%. 1 295 review cả hai đều sai; đây là các review khó thật hoặc bị gán nhãn sai.

## 2. Câu tự viết (application)

| Câu | P(pos) scratch | P(pos) fine-tuned | Nhận xét |
|---|---|---|---|
| This movie was absolutely wonderful, I loved every minute. | 1.000 | 0.996 | cả hai đúng |
| Not bad at all, actually one of the better films this year. | **0.021** | 0.942 | **negation**: scratch thấy "bad" là đoán negative; fine-tuned hiểu "not bad" |
| I expected to hate it, but it turned out to be great. | 0.998 | 0.940 | cả hai đúng |
| It was great... if you enjoy falling asleep. | 0.996 | 0.973 | **sarcasm**: cả hai sai, tin vào "great" |
| The acting was fine but the plot made no sense whatsoever. | 0.001 | 0.009 | cả hai đúng (negative) |
| A masterpiece of boredom. | 0.995 | 0.853 | sarcasm: cả hai sai; fine-tuned bớt tự tin hơn |
| Oh great, another two hours of my life I will never get back. | 0.999 | 0.712 | sarcasm: cả hai sai; fine-tuned gần ngưỡng |
| The food was cold and the waiter was rude. | 0.237 | 0.045 | domain khác (nhà hàng): fine-tuned chắc chắn hơn |
| Battery life is excellent and it charges quickly. | 1.000 | 0.943 | domain khác (sản phẩm): cả hai đúng |
| Phim rất hay, diễn viên đóng tuyệt vời. | 0.966 | 0.647 | tiếng Việt: tokenizer bỏ dấu, fine-tuned không chắc |

Pretraining giúp rõ nhất ở **negation** ("not bad"), vì nghĩa phụ thuộc ngữ cảnh. Với **sarcasm** thì cả hai đều sai: từ tích cực bề mặt vẫn thắng. Dù vậy, model fine-tuned ít tự tin hơn (0.71–0.85 so với 0.995–0.999).

## 3. Các mẫu sai

| # | Text (rút gọn) | True | Pred | Confidence | Error type | Hypothesis |
|---|---|---|---|---|---|---|
| **FP** 4116 | "...he will never win an Oscar... the geography was wrong... However, there are worse ways to spend an hour and a half. So grab your Doritos... and enjoy a trip back in time." | neg | pos | 0.655 | ambiguous sentiment | Câu kết mang giọng khuyên xem ("enjoy"), nên nhãn negative chỉ dựa vào phần chê ở giữa. Bản thân review lưng chừng, model không chắc (0.66). |
| **FP** 10188 | "John Wayne is without a doubt one of the most popular and loved actors... films hailed as some of his best, unlike this 1934 effort... nothing memorable..." | neg | pos | 0.889 | long-range + truncation | Mở đầu là một đoạn dài khen John Wayne. Phần chê phim chỉ có một mệnh đề "unlike...", và review 512 token nên phần kết luận ở nửa sau bị cắt mất ở T = 256. |
| **FP** 11985 | "Go to the video store and get the original... 'Mostly Martha' has a much better cast... convinced me much more..." | neg | pos | 0.656 | context (so sánh) | Lời khen dành cho phim gốc, không phải phim đang review. Model gom lời khen mà không xác định được nó nói về đối tượng nào. |
| **FP** 8793 | "...this drama is a bit comedic... change that to zany. Not being a MacDowell fan... albeit I enjoyed the film and it is not a total waste." | neg | pos | 0.869 | label ambiguity | Người viết nói "I enjoyed the film". Nhãn negative (rating ≤ 4) mâu thuẫn với nội dung chữ. |
| **FN** 16685 | "Gary Busey saved this film... would have been sub par. I recommend the film but just barely. The biggest difficulty... broad disregard for historical... accuracy. Why is... Why are..." | pos | neg | 0.977 | ambiguous sentiment | Có 1 câu khen và khoảng 8 câu hỏi chất vấn liên tiếp. Model đếm phần chê chiếm đa số, trong khi kết luận ("I recommend") là tích cực. |
| **FN** 19513 | "...entertainingly atrocious nickel'n'dime shot-on-video clunker... hilariously horrendous hoot... gloriously gross'n'graphic gore." | pos | neg | 0.815 | domain-specific expression | Kiểu review phim "so bad it's good": từ chê ("atrocious", "horrendous") được dùng để khen. Từ ngữ này đặc thù của fan phim kinh dị hạng B. |
| **FN** 23886 | "if you are dating a girl that is into wicca! many parts of this movie were killer... but some things really bothered me... you really need to be on a lot of drugs to get just what is going on" | pos | neg | 0.538 | ambiguous sentiment + informal | Viết không dấu câu, khen và chê đan xen. "killer" là từ lóng mang nghĩa tốt. Model gần ngưỡng (0.54). |
| **FN** 19636 | "...could be one of the most gretatest films in tne mexican movie history... sordidness, black humor... incestual..." | pos | neg | 0.543 | rare word / tokenization | Lỗi chính tả "gretatest", "tne" bị cắt thành mảnh vụn. Nhiều từ mô tả nội dung u ám ("sordidness", "violence") dễ bị hiểu là đang chê. |
| **Tự tin, sai** 12061 | "This movie was pure genius... hilarious... I give it 9.5/10. Rent it today!" | neg | pos | 0.996 | label ambiguity (label noise) | Review khen rõ ràng và tự cho 9.5/10, nhưng nhãn lại là negative. Model đúng, nhãn sai. Review này cũng là lỗi tự tin nhất ở LAB 07. |
| **Tự tin, sai** 4118 | "This is definitely one of the best Kung fu movies in the history of Cinema... The final fight... is a masterpiece!" | neg | pos | 0.996 | label ambiguity (label noise) | Không có câu chê nào, gần như chắc chắn là gán nhãn sai. |
| **Tự tin, sai** 2688 | "I really liked this quirky movie... captivating story line... fun to watch." | neg | pos | 0.995 | label ambiguity (label noise) | Tương tự: toàn bộ là khen. |
| **Tự tin, sai** 19650 | "...it was MUCH worse. First of all, there's the music. It's just plain awful... On the other hand, Bill Murray is spot on brilliant... turning a weak script into comic genius." | pos | neg | 0.995 | long-range + truncation | Nửa đầu chê nhạc rất nặng; lời khen Bill Murray và kết luận nằm ở nửa sau. Review dài 512 token, bị cắt ở T = 256 nên model không thấy phần kết. |

**Phân loại nguyên nhân**

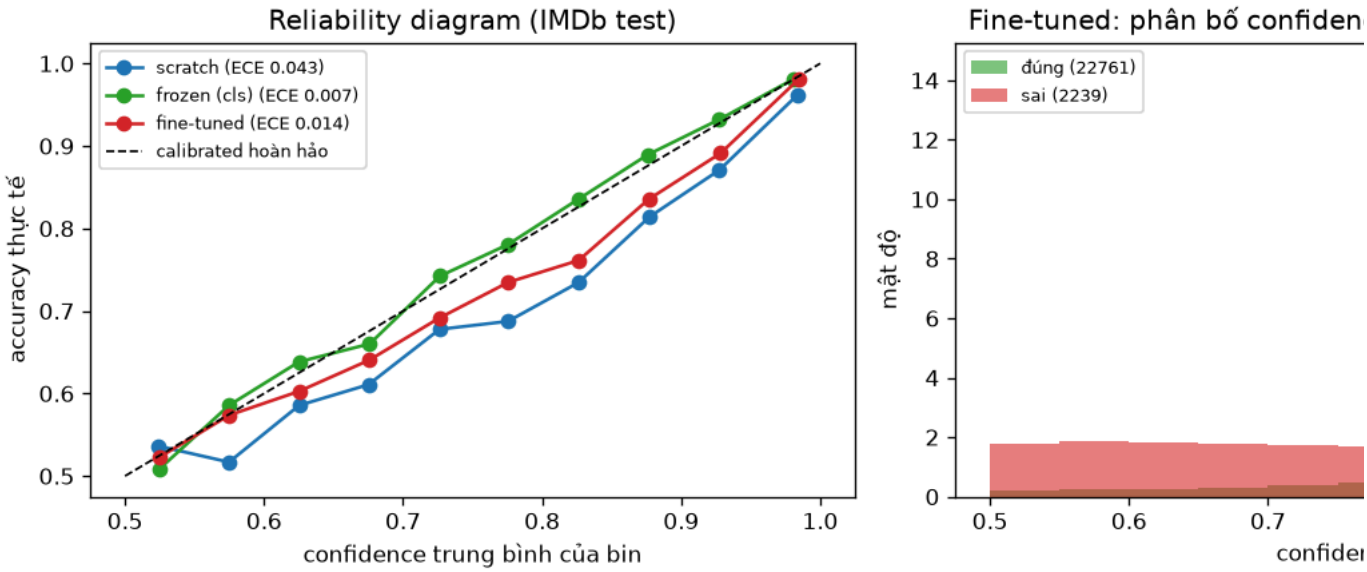
| Nhóm lỗi | Mẫu | Ghi chú |
|---|---|---|
| Label ambiguity / label noise | 12061, 4118, 2688, 8793 | 3 trên 4 lỗi tự tin nhất là review khen rõ ràng nhưng mang nhãn negative. Ở đây model không sai. |
| Long-range dependency + truncation | 10188, 19650 | Kết luận nằm sau token 256. Tỉ lệ lỗi của review bị cắt gấp đôi review không bị cắt. |
| Ambiguous / mixed sentiment | 4116, 16685, 23886 | Review vừa khen vừa chê, nhãn dựa vào điểm số chứ không dựa vào số câu khen / chê. |
| Context (đối tượng được khen) | 11985 | Khen phim khác để chê phim đang review. |
| Domain-specific expression / sarcasm | 19513 (+ các câu sarcasm ở mục 2) | Từ chê dùng với nghĩa khen. |
| Rare word / tokenization | 19636 | Lỗi chính tả bị cắt thành subword vô nghĩa. |

Không có lỗi nào do **negation** đơn giản. Model fine-tuned xử lý đúng "not bad", "not a total waste", trong khi model from scratch sai ở "Not bad at all".

## 4. Confidence analysis

`confidence = max(softmax(logits))`. Ba nhóm: High ≥ 0.95, Medium 0.75–0.95, Low < 0.75.

| Model | Nhóm | % mẫu | Accuracy | Confidence TB | % tổng lỗi |
|---|---|---|---|---|---|
| Fine-tuned (ECE 0.014) | High | 67.4% | **0.981** | 0.985 | 14.2% (317 lỗi) |
| | Medium | 22.2% | 0.835 | 0.878 | 40.9% |
| | Low | 10.4% | 0.614 | 0.635 | 44.9% |
| From scratch (ECE 0.043) | High | 45.8% | 0.962 | 0.984 | 10.4% |
| | Medium | 33.7% | 0.801 | 0.871 | 39.8% |
| | Low | 20.5% | 0.591 | 0.632 | 49.8% |
| Frozen [CLS] (ECE 0.007) | High | 35.0% | 0.981 | 0.981 | 4.6% |
| | Medium | 40.2% | 0.877 | 0.869 | 33.6% |
| | Low | 24.8% | 0.632 | 0.630 | 61.9% |




**Confidence cao có đồng nghĩa với prediction đúng không? Không.**

- **Quan sát:** nhóm High đúng 98.1%, nhưng vẫn có 317 lỗi, tức 14.2% tổng số lỗi. 1.2% số lỗi có confidence > 0.99.
- **Bằng chứng:** 4 lỗi tự tin nhất (≥ 0.995) gồm 3 review gán nhãn sai và 1 review mà phần kết luận bị cắt mất. Model "chắc chắn" dựa trên phần văn bản nó nhìn thấy; nó không biết phần bị cắt hay nhãn sai.
- **Diễn giải:**
- Confidence chỉ phản ánh mức nhất quán của tín hiệu trong input. Nó không đo được thông tin bị thiếu (truncation) hay lỗi của nhãn.
- Trên toàn tập, model fine-tuned khá calibrated (ECE 0.014): accuracy từng nhóm gần bằng confidence trung bình. Model from scratch tự tin quá mức (nhóm Medium có confidence 0.871 nhưng chỉ đúng 0.801).
- Frozen calibrated tốt nhất (ECE 0.007) nhưng ít khi tự tin: chỉ 35% mẫu nằm trong nhóm High.
- Trên domain khác, confidence trung bình của model fine-tuned vẫn khoảng 0.90–0.92 (Exp 5). Tức confidence không báo hiệu khi dữ liệu thay đổi domain.

# Reflection — LAB 08

## 1. Kết quả chính (IMDb test, 25 000 review)

| Model | Accuracy | F1 | Training time | # Parameters (train được) |
|---|---|---|---|---|
| Transformer from scratch (3 seed) | 0.835 ± 0.004 | 0.835 ± 0.003 | 33 s | 4 039 554 |
| Frozen DistilBERT + head ([CLS]) | 0.853 | 0.850 | 290 s* | 1 538 |
| Frozen DistilBERT + head (mean) | 0.859 | 0.856 | 289 s* | 1 538 |
| **DistilBERT fine-tuned (3 seed)** | **0.911 ± 0.002** | **0.911 ± 0.002** | 484 s | 66 364 418 |

* Gần như toàn bộ là thời gian chạy encoder một lần để lấy feature (train 223 s + val 56 s); train head chỉ mất khoảng 10 s.

## 2. Đối chiếu prediction

| Prediction | Dự đoán | Thực tế | Đúng? |
|---|---|---|---|
| P1: pretrained > scratch | 0.90–0.92 so với 0.83–0.85, chênh ≥ 5 điểm | 0.911 so với 0.835 (chênh 7.6 điểm); val loss epoch 1 đã là 0.238, thấp hơn mức tốt nhất của scratch (0.37) | Đúng |
| P2: frozen | C. kém hơn, khoảng 0.80–0.85 | 0.853 / 0.859, kém fine-tuning 5–6 điểm | Đúng |
| P3: 10% data | scratch giảm mạnh hơn | scratch giảm 12.4 điểm, pretrained 3.5 điểm | Đúng; pretrained ở 10% (0.876) cao hơn mức dự đoán |
| Phụ: max length | tăng theo T, mức tăng nhỏ dần | 0.822 → 0.875 → 0.910: mức tăng **không** nhỏ dần nhiều (+5.3 rồi +3.5) | Một phần |
| Phụ: domain shift | giảm còn 0.85–0.88 | **không giảm**: Amazon 0.927, Yelp 0.924, cao hơn IMDb 0.910. Model scratch giảm 3.6–4.7 điểm | Sai với pretrained |

Prediction về domain shift sai vì mình chỉ nghĩ tới chênh lệch từ vựng. Mình bỏ qua hai yếu tố khác:

- Review Yelp và Amazon **ngắn** (median 125 và 94 token), nên không bị cắt ở T = 256. Trên IMDb, 41% review bị cắt.
- Review Yelp và Amazon nói cảm xúc **trực tiếp hơn** review phim, vốn hay kể lại cốt truyện.

## 3. Trả lời câu hỏi các experiment

### Exp 1 — Pretrained vs from scratch

1. **Representation:** - Model from scratch học embedding và attention chỉ từ 20 000 review. Mỗi token chỉ là "dấu hiệu tích cực hay tiêu cực" của IMDb, không có nghĩa nào khác. - DistilBERT mang theo contextual representation học từ corpus lớn. Cùng một token có vector khác nhau tùy ngữ cảnh. Vì vậy nó xử lý đúng "Not bad at all" (0.94), trong khi model from scratch cho 0.02.
2. **Training data:** - Model from scratch cần nhiều dữ liệu, vì phải học cả nghĩa từ. - Pretrained chỉ cần học ánh xạ sang nhãn. Bằng chứng: chỉ với 10% data (1 992 review), pretrained đạt 0.876, cao hơn model from scratch dùng 100% data (0.835).
3. **Chi phí tính toán:** - Pretrained lớn hơn 16 lần (66.4 triệu tham số so với 4.0 triệu) và train lâu hơn khoảng 15 lần (484 s so với 33 s). - Pretrained dùng 1.95 GB VRAM, so với 0.16 GB. Dự đoán trên 25 000 review cũng chậm hơn nhiều. - Phần chi phí lớn nhất, tức pretraining, đã có người khác trả trước.
4. **Khớp prediction:** có (xem bảng mục 2).

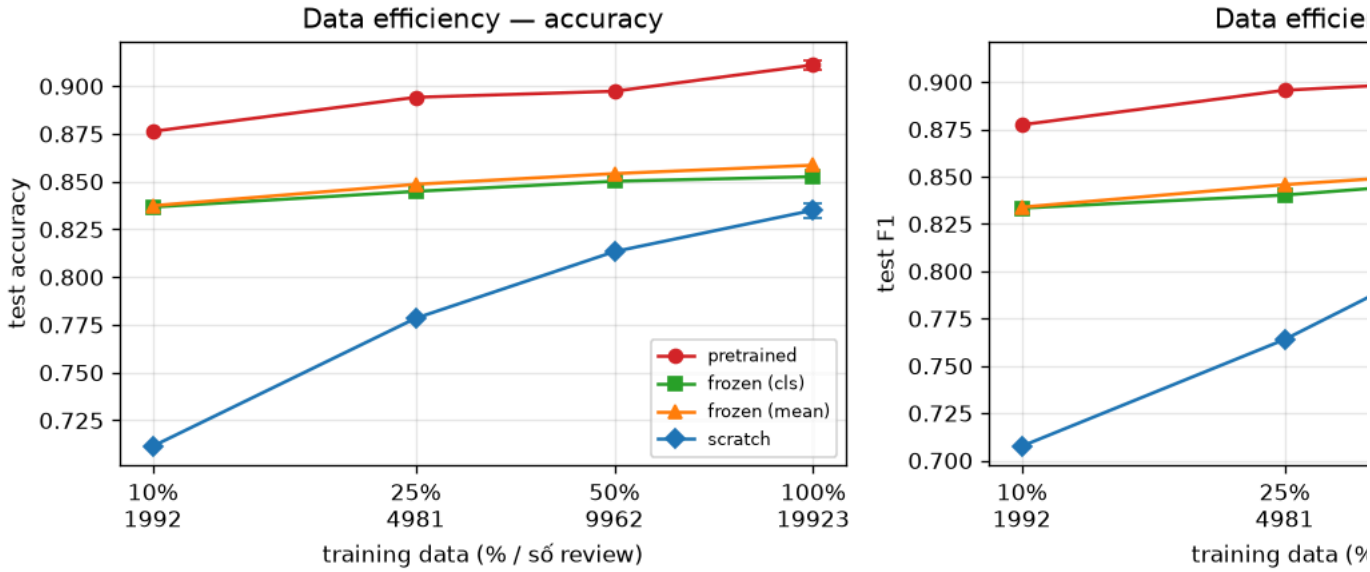
### Exp 2 — Frozen vs fine-tuning

| Mode | Encoder trainable? | Accuracy | F1 | Training time |
|---|---|---|---|---|
| Frozen ([CLS]) | No | 0.853 | 0.850 | 290 s |
| Frozen (mean) | No | 0.859 | 0.856 | 289 s |
| Fine-tuning | Yes | 0.911 | 0.911 | 484 s |

- **Vì sao frozen vẫn phân loại được:**
- Pretraining bằng MLM buộc encoder phải mã hóa nghĩa của câu, và sentiment là một phần của nghĩa đó. Ở `tokenization.ipynb`, model MLM chưa từng train sentiment vẫn điền "amazing" sau "I loved every minute" và "boring" sau "I walked out".
- Chỉ với 1 538 tham số (một Linear), frozen đã vượt model from scratch 4 triệu tham số.
- **Vì sao fine-tuning tốt hơn:**
- Representation của encoder frozen được tối ưu cho MLM, không phải cho sentiment. Thông tin sentiment có trong đó nhưng không tách được hoàn toàn bằng một siêu phẳng tuyến tính.
- Fine-tuning điều chỉnh mọi layer để dồn thông tin sentiment vào vector [CLS]. Với DistilBERT (không có pretraining NSP), vector [CLS] chưa từng được train để tóm tắt câu, vì vậy frozen [CLS] kém frozen mean một chút.

### Exp 3 — Lượng training data

| Training data | Pretrained acc / F1 | Frozen (mean) acc | Scratch acc / F1 |
|---|---|---|---|
| 10% (1 992) | 0.876 / 0.877 | 0.837 | 0.712 / 0.708 |
| 25% (4 981) | 0.894 / 0.896 | 0.849 | 0.778 / 0.764 |
| 50% (9 962) | 0.897 / 0.901 | 0.854 | 0.813 / 0.817 |
| 100% (19 923) | 0.911 / 0.911 | 0.859 | 0.835 / 0.835 |



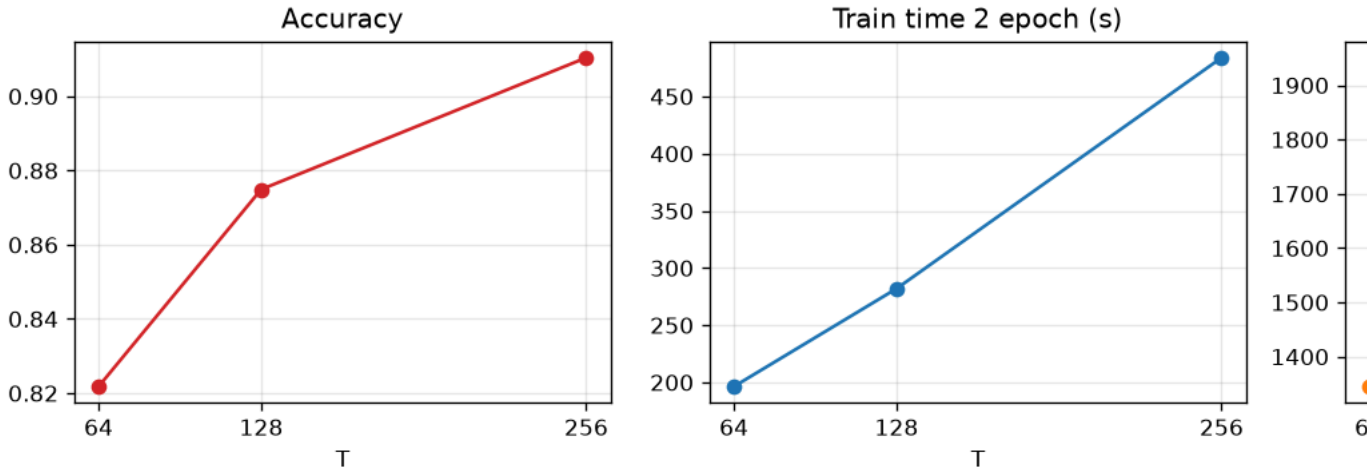

**Pretraining có giúp dùng dữ liệu downstream hiệu quả hơn không? Có.** Kết luận này dựa trên cả đường cong, không phải một điểm:

- Ở **mọi** mức data, pretrained luôn cao hơn scratch. Khoảng cách lớn nhất ở 10% (16.5 điểm) và thu hẹp dần: 11.6 → 8.4 → 7.6 điểm.
- Đường của pretrained **phẳng**: từ 10% lên 100% chỉ tăng 3.5 điểm. Đường của scratch **dốc** (+12.4 điểm) và vẫn chưa bão hòa ở 100%.
- Frozen phẳng nhất (+2.1 điểm), vì chỉ học 1 538 tham số. Thêm data không giúp được nhiều khi representation cố định.

Lưu ý: các điểm 10–50% chỉ chạy 1 seed. Độ lệch giữa 3 seed ở 100% là ±0.002 (pretrained) và ±0.004 (scratch), nhỏ hơn nhiều so với các khoảng cách trên.

### Exp 4 — Max sequence length

| Max length | Accuracy | F1 | Training time (2 epoch) | Peak GPU memory | % review bị cắt |
|---|---|---|---|---|---|
| 64 | 0.822 | 0.821 | 196 s | 1 345 MB | 97.9% |
| 128 | 0.875 | 0.874 | 282 s | 1 548 MB | 87.8% |
| 256 | 0.910 | 0.910 | 484 s | 1 951 MB | 41.2% |




**Tăng sequence length có luôn làm model tốt hơn không? Không.**

- Ở đây tăng T giúp nhiều, vì review IMDb dài (median 221 token): T = 64 chỉ thấy khoảng 1/4 review. Kết luận này phụ thuộc vào độ dài dữ liệu.
- Với Yelp hay Amazon (median 94–125 token), tăng T quá 128 gần như không có thêm thông tin, chỉ thêm chi phí.
- Chi phí tăng **chậm hơn** $T^2$: T × 4 thì thời gian × 2.5 và bộ nhớ × 1.45.
- Ở T ≤ 256, phần tuyến tính theo T (FFN, các phép chiếu Q/K/V, embedding) vẫn chiếm phần lớn.
- Phần $O(T^2)$ của attention mới chỉ bắt đầu lộ ra. LAB 07 đã cho thấy ở T ≥ 512 nó chiếm ưu thế: T = 1 024 vượt 4 GB VRAM.
- DistilBERT còn bị giới hạn cứng ở 512 token (position embedding học sẵn), nên muốn đọc cả review dài cũng không được.

### Exp 5 — Domain shift

| Model | IMDb (in-domain) | Amazon | Yelp |
|---|---|---|---|
| Scratch | 0.832 | 0.785 (−4.7) | 0.796 (−3.6) |
| Frozen ([CLS]) | 0.853 | 0.874 (+2.1) | 0.875 (+2.2) |
| Frozen (mean) | 0.859 | 0.860 (+0.1) | 0.857 (−0.2) |
| Fine-tuned | 0.910 | 0.927 (+1.6) | 0.924 (+1.3) |

**Pretrained model có "hiểu mọi domain" không? Không, và kết quả này không chứng minh điều đó.**

- **Quan sát:** model fine-tuned không giảm trên Yelp và Amazon, còn model from scratch giảm 3.6–4.7 điểm.
- **Bằng chứng:**
- Model from scratch chỉ biết từ vựng IMDb. "food", "waiter", "battery" gần như không có trong dữ liệu train, nên nó chỉ dựa vào vài từ cảm xúc chung.
- DistilBERT đã thấy các domain này lúc pretrain (Wikipedia, sách).
- Nhưng Yelp và Amazon là domain **gần** (vẫn là review, cảm xúc nói thẳng) và **dễ hơn** (ngắn, không bị cắt).
- **Diễn giải:**
- Đây là trường hợp domain shift nhẹ. Nó cho thấy pretraining giúp chống domain shift về **từ vựng**, chứ không cho thấy model hiểu mọi domain.
- Với domain xa hơn (tin tài chính, văn bản y khoa, tweet nhiều tiếng lóng, tiếng Việt), không có bằng chứng nào ở đây cho thấy model vẫn tốt. Câu tiếng Việt ở mục application chỉ được P = 0.65.
- Frozen có recall rất cao nhưng precision thấp trên Yelp (0.919 / 0.845), tức là bị lệch về phía positive: ngưỡng quyết định học trên IMDb không còn đúng ở domain mới.

## 4. Câu hỏi bắt buộc (mục 28)

- **Q1. Model nào hưởng lợi nhiều hơn khi data ít?** Pretrained.
- Ở 10% data, pretrained hơn scratch 16.5 điểm. Ở 100%, khoảng cách chỉ còn 7.6 điểm.
- Khoảng cách càng lớn khi data càng ít, nghĩa là phần kiến thức pretraining mang lại thay thế được cho dữ liệu downstream.
- **Q2. Pretrained tốt hơn from scratch chứng minh điều gì?**
- Chứng minh: với **dữ liệu này** (20 000 review) và **ngân sách train này**, khởi tạo từ trọng số pretrained tốt hơn khởi tạo ngẫu nhiên.
- Không chứng minh model "hiểu ngôn ngữ". Hai model còn khác nhau cả về kích thước (66 triệu so với 4 triệu tham số) và số layer (6 so với 1), nên một phần chênh lệch có thể đến từ dung lượng model.
- Phép thử frozen tách được hai yếu tố: không train encoder, chỉ 1 538 tham số, vẫn hơn scratch. Như vậy chính representation pretrained là yếu tố quan trọng.
- **Q3. Frozen gần bằng fine-tuned nghĩa là gì?**
- Nghĩa là representation pretrained đã chứa sẵn gần đủ thông tin cho task, chỉ cần một Linear đọc ra. Khi đó nên dùng frozen: rẻ hơn, không sợ quên kiến thức cũ, và một encoder dùng chung được cho nhiều task.
- Ở đây khoảng cách là 5–6 điểm, nên không phải trường hợp này.
- **Q4. Fine-tuning tốt hơn frozen nhiều gợi ý điều gì?**
- Task cần những đặc trưng mà MLM không trực tiếp tạo ra, ví dụ cân nhắc khen và chê trong cả review, hay hiểu "not bad". Encoder cần được điều chỉnh để đưa các đặc trưng này lên bề mặt.
- Điều đó cũng có nghĩa là các tham số pretrained bị thay đổi. Cần learning rate nhỏ (2e-5) và ít epoch: ở đây val loss đã tăng từ epoch 2 (0.238 → 0.279), dấu hiệu bắt đầu overfit hoặc quên kiến thức cũ.
- **Q5. Performance giảm mạnh trên domain khác có thể do đâu?**
- Từ vựng và cách nói khác nhau: từ chuyên ngành bị cắt vụn thành subword.
- Phân phối nhãn hoặc độ dài khác nhau.
- Cùng một từ mang sentiment khác nhau: "unpredictable" khen phim nhưng chê xe.
- Ngưỡng 0.5 học trên domain A không còn phù hợp (frozen lệch về positive trên Yelp).
- Representation pretrained không phủ domain đó: domain chuyên môn hoặc ngôn ngữ khác.
- **Q6. T tăng nhưng performance không đổi trong khi thời gian tăng mạnh: có nên tăng tiếp không?**
- Không. Phần văn bản thêm vào không mang thông tin mới cho task, trong khi chi phí tăng theo $O(T^2)$ về lâu dài.
- Nên chọn T nhỏ nhất mà accuracy đã bão hòa, dựa trên phân phối độ dài của dữ liệu. Nếu thông tin quan trọng nằm cuối văn bản, có thể giữ cả đầu lẫn cuối (head + tail truncation) thay vì tăng T.
- Ở IMDb, từ 128 lên 256 vẫn còn tăng 3.5 điểm, nên 256 vẫn đáng.

## 5. Individual learning check

1. **Pretraining khác fine-tuning ở đâu?** - Pretraining: train trên corpus rất lớn, không nhãn, với mục tiêu tự giám sát (MLM) để học representation chung. - Fine-tuning: bắt đầu từ trọng số pretrained, train tiếp trên dataset có nhãn của một task cụ thể, với learning rate nhỏ.
2. **MLM là gì?** Che ngẫu nhiên khoảng 15% token và bắt model đoán lại từ ngữ cảnh hai phía. Loss là cross-entropy trên các vị trí bị che.
3. **Tại sao cần tokenizer?** Model chỉ nhận số. Tokenizer chuyển text thành token ID theo **đúng** vocabulary lúc pretrain. Dùng sai tokenizer thì ID không khớp với embedding đã học.
4. **Subword giải quyết vấn đề gì?** - Vocabulary hữu hạn (30 000) nhưng không còn từ ngoài vocabulary: từ lạ được ghép từ mảnh, ví dụ `un ##watch ##able`. - Các từ cùng gốc chia sẻ mảnh, nên chia sẻ được thông tin.
5. **[CLS] dùng thế nào?** Token đặc biệt đặt ở đầu câu. Hidden state cuối của nó (qua self-attention đã nhìn mọi token) được dùng làm vector cho cả câu, rồi đưa vào Linear để phân loại.
6. **Attention mask dùng để làm gì?** Đánh dấu token thật (1) và `[PAD]` (0), để attention không nhìn vào padding. Bỏ mask thì vector [CLS] của cùng một câu đổi 6.68 (trên chuẩn 12.3) chỉ vì có thêm padding.
7. **Frozen encoder là gì?** Giữ nguyên trọng số encoder (`requires_grad = False`) và chỉ train head.
8. **Fine-tuning thay đổi tham số nào?** Toàn bộ: embedding, mọi layer attention và FFN, LayerNorm, cộng với head mới khởi tạo ngẫu nhiên.
9. **Vì sao pretrained hữu ích khi dataset nhỏ?** Phần lớn kiến thức (nghĩa từ, cú pháp, ngữ cảnh) đã có sẵn, nên dữ liệu ít chỉ cần dạy "hướng" của task. Ở 10% data: 0.876 so với 0.712.
10. **Pretrained có đảm bảo không bị domain shift không?** Không. Ở đây model giữ được trên Yelp và Amazon vì hai domain này gần và dễ. Domain xa hoặc ngôn ngữ khác vẫn có thể giảm mạnh, và confidence không báo trước điều đó.
11. **Vì sao tăng T làm chi phí attention tăng?** Mỗi token tính score với mọi token khác, nên ma trận attention có kích thước T × T cho mỗi head và mỗi layer. T tăng gấp đôi thì phần này tăng gấp 4 cả về phép tính lẫn bộ nhớ.
12. **Confidence cao có đảm bảo đúng không?** Không. Nhóm ≥ 0.95 vẫn có 317 lỗi, và 4 lỗi tự tin nhất là do nhãn sai hoặc phần kết luận bị cắt.

## 6. Reflection — Vì sao trong thực tế người ta không train Transformer từ đầu?

LAB 07 cho thấy tự xây một Transformer không khó: vài trăm dòng code, train 30 giây, accuracy 0.835. Nhưng LAB 08 cho thấy con số đó gần như là giới hạn của cách làm này, và lý do không nằm ở kiến trúc mà ở **dữ liệu**.

Model from scratch chỉ biết những gì có trong 20 000 review. Nó phải học cùng lúc nghĩa của 30 000 token và cách dùng chúng để đoán sentiment. Khi giảm xuống 2 000 review, accuracy rơi còn 0.71. Trong khi đó, DistilBERT với cùng 2 000 review vẫn đạt 0.876, cao hơn cả model from scratch dùng toàn bộ dữ liệu. Phần chênh lệch

đó là **pretrained representation**: kiến thức về từ, cú pháp và ngữ cảnh mà model đã học từ hàng tỉ token trước khi thấy IMDb. Thí nghiệm frozen cho thấy rõ nhất: không train encoder, chỉ thêm một lớp Linear 1 538 tham số, đã đạt 0.859.

Về **chi phí tính toán**, fine-tuning DistilBERT tốn khoảng 8 phút trên GPU laptop, gấp 15 lần model nhỏ. Đổi lại, phần đắt nhất, tức pretraining trên corpus khổng lồ, đã có người trả. Tự pretrain một model cỡ đó vượt xa khả năng của một sinh viên hay phần lớn công ty. Vì vậy cách làm hợp lý là dùng chung một model pretrained, rồi chỉ fine-tune cho từng **downstream task**. Head phân loại là phần duy nhất gắn với task; đổi task (sentiment, phân loại tin tức, NER) chỉ cần đổi head và dữ liệu nhãn.

**Fine-tuning** cũng không phải cứ train thêm là tốt. Learning rate phải nhỏ (2e-5). Ở đây val loss đã tăng từ epoch 2, và epoch tốt nhất là epoch 1. Train quá nhiều thì model overfit và có thể quên kiến thức pretrained.

Cuối cùng là **giới hạn**: - Model không đọc được quá 512 token, và lỗi tập trung ở review bị cắt (tỉ lệ lỗi 12.4% so với 6.7%). - Tokenizer cố định nên làm hỏng tiếng Việt và emoji. - Model vẫn thua sarcasm. - Trên Yelp và Amazon model không bị **domain shift**, nhưng đó là domain gần và dễ. Kết quả này không chứng minh model đã "hiểu mọi domain". Confidence trung bình vẫn khoảng 0.90 dù domain đã đổi, nên model cũng không tự báo được khi nó ra khỏi vùng quen thuộc.

Pretrained Transformer vì vậy là điểm xuất phát tốt nhất hiện nay, nhưng vẫn cần đánh giá trên dữ liệu thật của từng ứng dụng.

## 7. Bridge to LAB 09

Classification chỉ cho một nhãn cho cả review. Nhưng representation của DistilBERT có ở **mọi** vị trí token, không chỉ ở [CLS]. LAB 09 sẽ đặt head lên từng token (token classification) để gán nhãn PERSON, LOCATION, … cho NER.

Khi đó subword trở thành vấn đề thực tế: "Schwarzenegger" thành `schwarz ##ene ##gger`, vậy nhãn gán cho mảnh nào? Attention mask và truncation cũng phải xử lý ở mức từng token.